

Министерство образования и науки Российской Федерации
Санкт-Петербургский Политехнический Университет Петра Великого

—
Институт компьютерных наук и кибербезопасности

ЛАБОРАТОРНАЯ РАБОТА № 1
«РАСПРЕДЕЛЕННАЯ СИСТЕМА СБОРА ИНФОРМАЦИИ»
по дисциплине «БСИТ»

Выполнил
студент гр.5151001/40202

<подпись>

Волошкевич М.А.

Преподаватель /
ассистент

<подпись>

Дахнович А.Д.

Санкт-Петербург
2026г.

1. Цель работы.

Получить навыки организации параллельной обработки большого количества клиентских соединений при помощи механизма портов завершения Win32. Получить навыки реализации шифрования передаваемых данных при помощи средств CryptoAPI. Получить навыки извлечения информации о системе.

2. Архитектура

2.1. Выбор архитектуры системы

Разрабатываемая система предназначена для автоматического сбора информации обо всех компьютерах сети на одном центральном компьютере. На каждый опрашиваемый компьютер внедряется агент, представляющий собой серверную часть системы, а на центральном компьютере запускается клиентская часть. При проектировании необходимо было определить, какая из сторон инициирует передачу информации, какая из сторон работает постоянно и должен ли сервер хранить состояние между запросами. Принятые решения и их обоснование приведены в Таблица 1.

Вопрос	Решение	Обоснование
Кто инициирует передачу информации	Клиент	Центральный компьютер сам определяет, когда и какие компьютеры опрашивать. Агенту не нужно знать адрес центра, хранить расписание и очередь неотправленных данных. Автоматический режим сводится к запуску запросов клиента по списку адресов. Недостаток подхода: на каждом агенте должен быть доступен входящий порт.
Кто постоянно работает и принимает запрос	Сервер (агент)	Агент должен быть готов ответить в любой момент, поэтому он работает постоянно как консольное приложение без взаимодействия с пользователем. Клиент запускается на центральном компьютере только на время опроса.
Stateless или statefull	Stateless	Каждый запрос самодостаточен: одно подключение, один запрос, один ответ, после чего соединение закрывается. Сервер не хранит информацию о клиентах между запросами, поэтому не возникает проблем с оборванными сессиями и перезапуском агента, а обработка запросов легко распараллеливается. Единственное состояние, существующее в рамках одного подключения, это сеансовый ключ шифрования.

Таблица 1. Принятые архитектурные решения

2.2. Схема взаимодействия клиента и сервера

Взаимодействие строится поверх TCP (порт 9000). Каждый запрос выполняется в отдельном подключении и состоит из установки сеансового ключа и обмена одним зашифрованным запросом и одним зашифрованным ответом.

№	Направление	Содержимое	Защита
1	Клиент → сервер	Установка TCP-соединения	—
2	Клиент → сервер	Публичный ключ RSA-2048 клиента	Открытый вид
3	Сервер → клиент	Сеансовый ключ AES-256	Зашифрован публичным ключом клиента
4	Клиент → сервер	Запрос	AES-256, сеансовый ключ
5	Сервер → клиент	Ответ с запрошенной информацией	AES-256, сеансовый ключ
6	Сервер	Закрытие соединения	—

Таблица 2. Этапы взаимодействия клиента и сервера

2.3. Прикладной протокол

2.3.1. Формат сообщения

TCP передаёт поток байт и не сохраняет границы сообщений, поэтому каждое сообщение, как до установки сеансового ключа, так и после, передаётся в виде кадра. Кадр состоит из четырёх байт длины тела в сетевом порядке байт и самого тела. Получатель сначала читает заголовок, затем ровно указанное число байт, что исключает проблемы с сообщениями, пришедшими частями или склеенными. Размер кадра ограничен (сервер принимает не более 16 КБ), что защищает от некорректных клиентов.

Поле	Размер	Описание
Длина	4 байта	Длина тела кадра, big endian
Тело	Длина байт	Публичный ключ, сеансовый ключ или зашифрованное сообщение

Таблица 3. Формат кадра

2.3.2. Запросы

Для передачи каждого типа информации определён отдельный запрос. Запрос представляет собой одну строку в кодировке UTF-8: имя команды и, если нужно, аргументы, разделённые символом табуляции.

Запрос	Аргументы	Информация в ответе
GET_OS	нет	Тип и версия ОС
GET_TIME	нет	Текущее время (UTC и местное)
GET_UPTIME	нет	Время, прошедшее с момента запуска ОС
GET_MEMORY	нет	Информация об используемой памяти
GET_DRIVES	нет	Типы подключённых дисков и файловые системы
GET_FREESPACE	нет	Свободное место на локальных дисках
GET_ACL	тип объекта (FILE или REG), путь	Права доступа к файлу, папке или ключу реестра
GET_OWNER	тип объекта (FILE или REG), путь	Владелец файла, папки или ключа реестра

Таблица 4. Запросы протокола

2.3.3. Формат ответа

Формат ответа формализован и пригоден для машинной обработки. Ответ это текст в кодировке UTF-8, состоящий из строк, каждая из которых является записью вида ТЕГ, затем пары ключ=значение, разделённые табуляцией. Первая запись любого ответа это RESULT с признаком успеха. Числовые значения (байты, миллисекунды, номера битов) передаются в исходном виде без форматирования, а оформление для пользователя выполняет клиент. В значениях экранируются обратная косая черта, табуляция и перевод строки, поэтому пути и имена с любыми символами не нарушают разбор.

Пример ответа на запрос GET_OS. Символ табуляции показан как <TAB>; для наглядности поля записи разнесены по строкам (продолжения начинаются с отступа), в действительности каждая запись занимает одну строку.

```
RESULT<TAB>status=OK<TAB>command=GET_OS
OS<TAB>name=Windows 10<TAB>product_name=Windows 10 Pro
  <TAB>display_version=<TAB>major=10<TAB>minor=0
  <TAB>build=19043<TAB>platform_id=2
  <TAB>service_pack=<TAB>sp_major=0<TAB>sp_minor=0
  <TAB>product_type=workstation<TAB>arch=x64
  <TAB>computer_name=VM
```

Ответы с несколькими однотипными объектами содержат по записи на каждый объект. Например, ответ на GET_ACL состоит из записи ACL с общими сведениями и записей ACE по числу элементов списка (показана первая запись, список битов сокращён):

```
RESULT<TAB>status=OK<TAB>command=GET_ACL
ACL<TAB>object_type=directory<TAB>path=C:\\windows
  <TAB>null_dacl=0<TAB>dacl_protected=0<TAB>ace_count=3
ACE<TAB>index=0<TAB>sid=S-1-5-18
  <TAB>name=NT AUTHORITY\\SYSTEM<TAB>name_use=well_known_group
  <TAB>ace_type=ACCESS_ALLOWED_ACE_TYPE<TAB>ace_type_id=0
  <TAB>ace_flags=0x00000003
  <TAB>ace_flag_names=OBJECT_INHERIT_ACE,CONTAINER_INHERIT_ACE
  <TAB>scope=this_folder_subfolders_and_files
```

```

<TAB>mask=0x001F01FF
<TAB>bits=0,1,2,3,4,5,6,7,8,16,17,18,19,20
<TAB>bit_names=FILE_LIST_DIRECTORY,FILE_ADD_FILE,...

```

В пути и имени обратная косая черта удвоена из-за экранирования.

Запись	Поля
RESULT	status (OK или ERROR), command; при ошибке code, source, message
OS	name, product_name, display_version, major, minor, build, platform_id, service_pack, sp_major, sp_minor, product_type, arch, computer_name
TIME (две записи)	scope (utc или local), iso, year, month, day, hour, minute, second, millisecond, day_of_week, unix; для местного времени ещё utc_offset_minutes, tz_name
UPTIME	milliseconds, boot_time_utc, boot_time_unix
MEMORY	load_percent, total_phys, avail_phys, total_pagefile, avail_pagefile, total_virtual, avail_virtual
DRIVE	root, type, class, fs, label, ready
FREESPACE	root, type, total_bytes, free_bytes, free_to_user_bytes или error
OWNER	object_type, path, sid, name, name_use
ACL	object_type, path, null_dacl, dacl_protected, ace_count
ACE	index, sid, name, name_use, ace_type, ace_type_id, ace_flags, ace_flag_names, scope, mask, bits, bit_names

Таблица 5. Записи и поля ответа

Если запрос не удалось выполнить, ответ состоит из одной записи RESULT со статусом ERROR. Поле source показывает источник ошибки: protocol для ошибок протокола (неизвестная команда, неверные аргументы) и win32 для ошибок системных функций. В последнем случае передаются код ошибки и её текст, полученный от системы.

RESULT<TAB>status=ERROR<TAB>code=5<TAB>source=win32<TAB>message=Отказано в доступе.

RESULT<TAB>status=ERROR<TAB>code=1<TAB>source=protocol<TAB>message=unknown
command

```
--- сырой ответ ---  
RESULT status=OK      command=GET_OS  
OS      name=Windows 10 product_name=Windows 10 Pro display_version=22H2 major=10 minor=0  
build=19045 platform_id=2 service_pack= sp_major=0 sp_minor=0 product_type=workstation  
arch=x64 computer_name=ARCHLINUX  
--- конец ---
```

Рис. 1. Сырой ответ сервера на запрос GET_OS (режим вывода сырого ответа клиента)

```
--- сырой ответ ---  
RESULT status=OK      command=GET_ACL  
ACL      object_type=directory path=C:\\demo null_dacl=0 dacl_protected=0 ace_count=3  
ACE      index=0 sid=S-1-5-18 name=NT AUTHORITY\\SYSTEM name_use=well_known_group ace_type  
=ACCESS_ALLOWED_ACE_TYPE ace_type_id=0 ace_flags=0x00000003 ace_flag_names=OBJECT_INHERIT_AC  
E  
CONTAINER_INHERIT_ACE scope=this_folder_subfolders_and_files mask=0x001F01FF bits=0,1,2,3,4,5,6,7,8,1  
6,17,18,19,20 bit_names=FILE_LIST_DIRECTORY,FILE_ADD_FILE,FILE_ADD_SUBDIRECTORY,FILE_READ_EA,FILE_WRI  
T  
_EA,FILE_TRAVERSE,FILE_DELETE_CHILD,FILE_READ_ATTRIBUTES,FILE_WRITE_ATTRIBUTES,DELETE,READ_CONTROL,WRITE  
_DAC,WRITE_OWNER,SYNCHRONIZE  
ACE      index=1 sid=S-1-5-21-2525597665-4019535977-1000-1000 name=ARCHLINUX\\red name_use=user  
ace_type=ACCESS_ALLOWED_ACE_TYPE ace_type_id=0 ace_flags=0x00000003 ace_flag_names=OBJECT_IN  
H  
ERIT_ACE,CONTAINER_INHERIT_ACE scope=this_folder_subfolders_and_files mask=0x001F01FF bits=0,1,2,3,4,5  
,6,7,8,16,17,18,19,20 bit_names=FILE_LIST_DIRECTORY,FILE_ADD_FILE,FILE_ADD_SUBDIRECTORY,FILE_READ_EA,F  
I  
LE_WRITE_EA,FILE_TRAVERSE,FILE_DELETE_CHILD,FILE_READ_ATTRIBUTES,FILE_WRITE_ATTRIBUTES,DELETE,READ_CONTR  
OL,WRITE_DAC,WRITE_OWNER,SYNCHRONIZE  
ACE      index=2 sid=S-1-1-0 name=Everyone name_use=well_known_group ace_type=ACCESS_ALLOWED_  
ACE_TYPE ace_type_id=0 ace_flags=0x00000003 ace_flag_names=OBJECT_INHERIT_ACE,CONTAINER_INHE  
R  
IT_ACE scope=this_folder_subfolders_and_files mask=0x001200A9 bits=0,3,5,7,17,20 bit_names=FILE_L  
IST_DIRECTORY,FILE_READ_EA,FILE_TRAVERSE,FILE_READ_ATTRIBUTES,READ_CONTROL,SYNCHRONIZE  
--- конец ---
```

Рис. 2. Сырой ответ сервера на запрос GET_ACL

```
--- сырой ответ ---  
RESULT status=ERROR code=2 source=win32 message=File not found.  
--- конец ---
```

Рис. 3. Ответ сервера с ошибкой

2.4. Шифрование передаваемых данных

Все сообщения между клиентом и сервером, кроме самого публичного ключа, передаются в зашифрованном виде с использованием CryptoAPI. Применена гибридная схема. Для шифрования данных используется симметричный алгоритм AES-256 с сеансовым ключом, а для первоначальной безопасной передачи этого ключа применяется асимметричный алгоритм RSA с ключом длиной 2048 бит. Криптопровайдер: Microsoft Enhanced RSA and AES Cryptographic Provider. Контекст получается с флагом CRYPT_VERIFYCONTEXT, контейнер ключей на диске не создаётся, все ключи существуют только в памяти процесса.

№	Шаг схемы	Функции CryptoAPI
1	Клиент генерирует пару ключей RSA (публичный и приватный). Пара создаётся один раз при запуске клиента	CryptGenKey
2	Клиент отправляет публичный ключ серверу	CryptExportKey (PUBLICKEYBLOB)
3	Сервер получает публичный ключ клиента	CryptImportKey
4	Сервер генерирует сеансовый ключ AES-256 для этого подключения	CryptGenKey (CALG_AES_256)
5	Сервер шифрует сеансовый ключ публичным ключом клиента и отправляет его	CryptExportKey (SIMPLEBLOB)
6	Клиент расшифровывает сеансовый ключ своим приватным ключом	CryptImportKey
7	Обе стороны шифруют и расшифровывают сообщения сеансовым ключом	CryptEncrypt, CryptDecrypt

Таблица 6. Схема шифрования с использованием сеансового ключа

Алгоритм AES работает в режиме CBC, данные дополняются до размера блока по схеме PKCS#5. В этом режиме одинаковые сообщения при одном ключе дали бы одинаковый шифртекст, поэтому для каждого сооб-

щения генерируется новый случайный вектор инициализации (16 байт, `CryptGenRandom`), который передаётся перед шифртекстом. Тело зашифрованного кадра имеет вид: вектор инициализации (16 байт), затем шифртекст.

Использование сеансового ключа даёт следующие преимущества:

- симметричные алгоритмы работают на порядки быстрее асимметричных;
- RSA способен шифровать только небольшой объём данных (для ключа 2048 бит около 245 байт), а ответы сервера бывают гораздо длиннее;
- не требуется заранее распространять общий секрет, ключ создаётся для каждого подключения и безопасно передаётся по открытому каналу;
- ключ одноразовый: компрометация одного ключа не раскрывает остальные сеансы, а под одним ключом шифруется ограниченный объём данных.

2.5. Анализ сетевого обмена в Wireshark

Для проверки протокола и шифрования был выполнен перехват трафика между клиентом и сервером 1 с помощью Wireshark. Захват запущен до подключения клиента, чтобы в него попала установка соединения.

1	0.000000000	10.10.0.10	10.10.0.11	TCP	74	55600 → 9000 [SYN] Seq=0 Win=64240 Len=0 MSS=1460 SACK_PERM TSval=2156149710 TSecr=0 WS=1024
2	0.000053546	0.000053546 10.10.0.11	10.10.0.10	TCP	74	9000 → 55600 [SYN, ACK] Seq=0 Ack=1 Win=65160 Len=0 MSS=1460 SACK_PERM TSval=2699507932 TSecr=2156149710 WS=1024
3	0.000083317	0.000029771 10.10.0.10	10.10.0.11	TCP	66	55600 → 9000 [ACK] Seq=1 Ack=1 Win=64512 Len=0 TSval=2156149710 TSecr=2699507932
4	0.0000997371	0.000014054 10.10.0.10	10.10.0.11	TPKT	70	Continuation
5	0.001025216	0.000027845 10.10.0.11	10.10.0.10	TCP	66	9000 → 55600 [ACK] Seq=1 Ack=5 Win=65536 Len=0 TSval=2699507933 TSecr=2156149711
6	0.001279084	0.000254408 10.10.0.10	10.10.0.11	TCP	342	55600 → 9000 [PSH, ACK] Seq=5 Ack=1 Win=64512 Len=276 TSval=2156149711 TSecr=2699507933
7	0.001303794	0.000024110 10.10.0.11	10.10.0.10	TCP	66	9000 → 55600 [ACK] Seq=1 Ack=281 Win=65536 Len=0 TSval=2699507933 TSecr=2156149711
8	0.003351546	0.002047752 10.10.0.11	10.10.0.10	TCP	338	9000 → 55600 [PSH, ACK] Seq=1 Ack=281 Win=65536 Len=272 TSval=2699507935 TSecr=2156149711
9	0.003381255	0.000029789 10.10.0.10	10.10.0.11	TCP	66	55600 → 9000 [ACK] Seq=281 Ack=273 Win=64512 Len=0 TSval=2156149713 TSecr=2699507935
10	0.020809264	0.017488009 10.10.0.10	10.10.0.11	TPKT	70	Continuation
11	0.061023725	0.040154401 10.10.0.11	10.10.0.10	TCP	66	9000 → 55600 [ACK] Seq=273 Ack=285 Win=65536 Len=0 TSval=2699507993 TSecr=2156149730
12	0.061060512	0.000041787 10.10.0.10	10.10.0.11	TCP	98	55600 → 9000 [PSH, ACK] Seq=285 Ack=273 Win=64512 Len=32 TSval=2156149771 TSecr=2699507993
13	0.061092097	0.000027185 10.10.0.11	10.10.0.10	TCP	66	9000 → 55600 [ACK] Seq=273 Ack=317 Win=65536 Len=0 TSval=2699507993 TSecr=2156149771
14	0.063574711	0.002482014 10.10.0.11	10.10.0.10	TCP	326	9000 → 55600 [PSH, ACK] Seq=273 Ack=317 Win=65536 Len=260 TSval=2699507995 TSecr=2156149771
15	0.063794953	0.000220242 10.10.0.11	10.10.0.10	TCP	66	9000 → 55600 [FIN, ACK] Seq=533 Ack=317 Win=65536 Len=0 TSval=2699507995 TSecr=2156149771
16	0.064097171	0.000302218 10.10.0.10	10.10.0.11	TCP	66	55600 → 9000 [FIN, ACK] Seq=317 Ack=534 Win=64512 Len=0 TSval=2156149774 TSecr=2699507995
17	0.064143301	0.000046130 10.10.0.11	10.10.0.10	TCP	66	9000 → 55600 [ACK] Seq=534 Ack=318 Win=65536 Len=0 TSval=2699507996 TSecr=2156149774

Рис. 4. Общий вид перехваченного трафика с фильтром по адресам клиента и сервера 1

Пакеты	Направление	Размер, байт	Содержимое
1–3	Клиент ↔ сервер	0	Установка TCP-соединения (SYN, SYN-ACK, ACK)
4	Клиент → сервер	4	Заголовок кадра с длиной публичного ключа (276)
6	Клиент → сервер	276	Публичный ключ клиента (PUBLICKEYBLOB, алгоритм RSA, 2048 бит)
8	Сервер → клиент	272	Заголовок кадра (268) и сеансовый ключ (SIMPLEBLOB), зашифрованный публичным ключом клиента
10, 12	Клиент → сервер	4 и 32	Заголовок кадра и зашифрованный запрос: вектор инициализации (16 байт) и один блок шифртекста
14	Сервер → клиент	260	Заголовок кадра (256) и зашифрованный ответ: вектор инициализации и шифртекст
15–17	Сервер ↔ клиент	0	Закрытие соединения по инициативе сервера

Таблица 7. Разбор перехваченного обмена

Рассмотрим этапы подробнее. После установки TCP-соединения клиент отправляет публичный ключ. На рисунке видно, что кадр начинается с заголовка ключа (06 02 00 00 00 A4 00 00), за которым следует сигнатура RSA1, длина ключа 2048 бит, открытая экспонента 65537 и 256 байт модуля. Ответом сервера служит сеансовый ключ: заголовок 01 02 00 00 10 66 00 00 00 A4 00 00 означает, что передаётся ключ алгоритма AES-256 (код 0x6610), зашифрованный алгоритмом RSA (0xA400), далее следуют 256 байт шифртекста, расшифровать которые может только владелец приватного ключа клиента. Дальнейший обмен содержит только зашифрованные данные: в запросе и ответе нет читаемого текста, а байты выглядят как случайные.

Размеры зашифрованных сообщений кратны размеру блока AES с добавлением 16 байт вектора инициализации.

1	0.000000000	10.10.0.10	10.10.0.11	TCP	74	55600 - 9000	[SYN]	Seq=0	Win=64240	Len=0	MSS=1460	SACK_PERM	Tsval=2156149710	Tsecr=0	WS=1024		
2	0.000053546	0.000053546	10.10.0.11	10.10.0.10	TCP	74	9000 - 55600	[SYN, ACK]	Seq=0	Ack=1	Win=65160	Len=0	MSS=1460	SACK_PERM	Tsval=2699507932	Tsecr=2156149710	WS=1024
3	0.000083317	0.000029771	10.10.0.10	10.10.0.11	TCP	66	55600 - 9000	[ACK]	Seq=1	Ack=1	Win=64512	Len=0	Tsval=2156149710	Tsecr=2699507932			

Рис. 5. Установка TCP-соединения между клиентом и сервером 1 (SYN, SYN-ACK, ACK)

1	0.000000000	0.000000000	10.10.0.10	10.10.0.11	TCP	74	55600	-	9000	[SYN]	Seq=0	Win=64240	Len=0	MSS=1460	SACK_PERM	Tsval=2156149710	Tsecr=0	WS=1024	
2	0.000053546	0.000053546	10.10.0.11	10.10.0.10	TCP	74	9000	-	55600	[SYN, ACK]	Seq=0	Ack=1	Win=65160	Len=0	MSS=1460	SACK_PERM	Tsval=2699507932	Tsecr=2156149710	WS=1024
3	0.000083317	0.000029771	10.10.0.10	10.10.0.11	TCP	66	55600	-	9000	[ACK]	Seq=1	Ack=1	Win=64512	Len=0	Tsval=2156149710	Tsecr=2699507932			
4	0.000097371	0.000014054	10.10.0.10	10.10.0.11	TPKT	70	Continuation												
5	0.001025216	0.000027045	10.10.0.11	10.10.0.10	TCP	66	9000	-	55600	[ACK]	Seq=1	Ack=5	Win=65536	Len=0	Tsval=2699507993	Tsecr=2156149711			
6	0.001270684	0.000254160	10.10.0.10	10.10.0.11	TCP	342	55600	-	9000	[PSH, ACK]	Seq=5	Ack=1	Win=64512	Len=276	Tsval=2156149711	Tsecr=2699507933			
7	0.001303794	0.000024110	10.10.0.11	10.10.0.10	TCP	66	9000	-	55600	[ACK]	Seq=1	Ack=281	Win=65536	Len=0	Tsval=2699507933	Tsecr=2156149711			
8	0.003351546	0.002047752	10.10.0.11	10.10.0.10	TCP	338	9000	-	55600	[PSH, ACK]	Seq=1	Ack=281	Win=65536	Len=272	Tsval=2699507935	Tsecr=2156149711			
9	0.003381255	0.000029709	10.10.0.10	10.10.0.11	TCP	66	55600	-	9000	[ACK]	Seq=281	Ack=273	Win=64512	Len=0	Tsval=2156149713	Tsecr=2699507935			
10	0.020869264	0.017488009	10.10.0.10	10.10.0.11	TPKT	70	Continuation												
11	0.001023725	0.000154461	10.10.0.11	10.10.0.10	TCP	66	9000	-	55600	[ACK]	Seq=273	Ack=285	Win=65536	Len=0	Tsval=2699507993	Tsecr=2156149730			
12	0.001065512	0.000041787	10.10.0.10	10.10.0.11	TCP	98	55600	-	9000	[PSH, ACK]	Seq=285	Ack=273	Win=64512	Len=32	Tsval=2156149771	Tsecr=2699507993			
13	0.001092697	0.000027185	10.10.0.11	10.10.0.10	TCP	66	9000	-	55600	[ACK]	Seq=273	Ack=317	Win=65536	Len=0	Tsval=2699507993	Tsecr=2156149771			
14	0.003574711	0.002482014	10.10.0.11	10.10.0.10	TCP	326	9000	-	55600	[PSH, ACK]	Seq=273	Ack=317	Win=65536	Len=260	Tsval=2699507995	Tsecr=2156149771			
15	0.003794953	0.000220242	10.10.0.11	10.10.0.10	TCP	66	9000	-	55600	[FIN, ACK]	Seq=533	Ack=317	Win=65536	Len=0	Tsval=2699507995	Tsecr=2156149771			
16	0.004097171	0.000302218	10.10.0.10	10.10.0.11	TCP	66	55600	-	9000	[FIN, ACK]	Seq=317	Ack=534	Win=64512	Len=0	Tsval=2156149774	Tsecr=2699507995			
17	0.004143301	0.000046130	10.10.0.11	10.10.0.10	TCP	66	9000	-	55600	[ACK]	Seq=534	Ack=318	Win=65536	Len=0	Tsval=2699507996	Tsecr=2156149774			

+ Frame 6: Packet, 342 bytes on wire (2736 bits), 342 bytes captured (2736 bits) on interface labbr, id 0

+ Ethernet II, Src: 10:00:75:c1:a1:b3 (10:00:75:c1:a1:b3), Dst: b2:80:38:be:95:00 (b2:80:38:be:95:00)

+ Internet Protocol Version 4, Src: 10.10.0.10, Dst: 10.10.0.11

+ Transmission Control Protocol, Src Port: 55600, Dst Port: 9000, Seq: 5, Ack: 1, Len: 276

- Data (276 bytes)

Data [..]: 0002000000a4000052534131000000000100010069e47bda5f93cc5e7204af0221cef9e94763ef309b5effc72d.

[Length: 276]

0000

b2 80 38 be 95 00 10 00

75 c1 a1 b3 80 00 45 00

8 0 0 0 0 0 0 0

0010

01 48 3a 87 40 00 40 00

eb 00 0a 0a 00 0a 0a

0 0 0 0 0 0 0 0

0020

00 00 49 30 23 28 c2 52

40 22 cd 77 fe 9b 00 18

0 0 0 0 0 0 0 0

0030

00 4f 15 03 00 00 01 01

00 0a 00 84 30 cf a0 07

0 0 0 0 0 0 0 0

0040

38 dd 00 00 02 00 00 00 a4

00 00 52 53 41 31 00 00

8 0 0 0 0 0 0 0

0050

00 00 01 00 00 01 00 69 e4

7b da f5 03 cc 5e 72 04

0 0 0 0 0 0 0 0

0060

af e2 21 ce f9 e0 47 63

ef 30 9b 5e 71 cf 2d f9

0 0 0 0 0 0 0 0

0070

a1 aa 4e 7f ca fb ca 0b

67 40 a5 57 9c 82 47 3f

0 0 0 0 0 0 0 0

0080

4f e2 94 c6 08 82 c5 ce

6c a5 e9 83 b1 82 60 cd

0 0 0 0 0 0 0 0

0090

da 26 00 bd e5 2d 92 87

0c 4c 9f 8d 9c e2 6b fd

0 0 0 0 0 0 0 0

00a0

90 fa 7f 7a 5d 0f 74 9e

d5 cc b5 57 3c 00 48 d3

0 0 0 0 0 0 0 0

00b0

90 90 84 01 6d d7 85 be

88 12 06 69 ba e2 97 87

0 0 0 0 0 0 0 0

00c0

cc f2 53 31 cf f9 57 75

79 12 e8 a6 f0 3f e7 81

0 0 0 0 0 0 0 0

00d0

de 71 fa ca e7 5c 82 31

2f 38 af 10 22 cc 60 12

0 0 0 0 0 0 0 0

00e0

e1 51 fa 28 0e d2 e2 1e

6f 04 6d 86 7e 08 8d df

0 0 0 0 0 0 0 0

00f0

20 e8 36 0b d9 ca df 10

4b df e5 22 39 70 d9 cc

0 0 0 0 0 0 0 0

0100

52 1f c1 23 4b 27 34 3a

32 04 50 a3 a3 b0 10 04

0 0 0 0 0 0 0 0

0110

f1 31 85 cc 08 71 40 ff

b9 92 ac 79 26 fc fb e8

0 0 0 0 0 0 0 0

0120

b5 2e 5b af 00 9a eb fd

88 fe 3f f1 57 f0 eb 98

0 0 0 0 0 0 0 0

0130

cc 47 cf 78 f1 9a 62 80

11 0c 38 4f e0 22 d3 68

0 0 0 0 0 0 0 0

0140

0e 70 ad bf b2 30 d4 0c

e6 00 7a bc d5 9a 33 c6

0 0 0 0 0 0 0 0

0150

10 a7 17 00 98 e4

Рис. 6. Пакет с публичным ключом клиента (поле данных, сигнатура RSA1)

1	0.000000000	0.10.10.0.10	10.10.0.11	TCP	74	55600 - 9000	[SYN]	Seq=0	Win=64240	Len=0	MSS=1460	SACK_PERM	Tsval=2156149710	Tsecr=0	WS=1024		
2	0.000053546	0.000053546	10.10.0.11	10.10.0.10	TCP	74	9000 - 55600	[SYN, ACK]	Seq=0	Ack=1	Win=65160	Len=0	MSS=1460	SACK_PERM	Tsval=2699507932	Tsecr=2156149710	WS=1024
3	0.000083317	0.000029771	10.10.0.10	10.10.0.11	TCP	66	55600 - 9000	[ACK]	Seq=1	Ack=1	Win=64512	Len=0	Tsval=2156149710	Tsecr=2699507932			
4	0.000097371	0.000014054	10.10.0.10	10.10.0.11	TPKT	70	Continuation										
5	0.001025216	0.000027845	10.10.0.11	10.10.0.10	TCP	66	9000 - 55600	[ACK]	Seq=1	Ack=5	Win=65536	Len=0	Tsval=2699507993	Tsecr=2156149711			
6	0.001270684	0.000254160	10.10.0.10	10.10.0.11	TCP	342	55600 - 9000	[PSH, ACK]	Seq=5	Ack=1	Win=64512	Len=276	Tsval=2156149711	Tsecr=2699507933			
7	0.001303794	0.000024110	10.10.0.11	10.10.0.10	TCP	66	9000 - 55600	[ACK]	Seq=1	Ack=281	Win=65536	Len=0	Tsval=2699507933	Tsecr=2156149711			
8	0.003351546	0.002047752	10.10.0.11	10.10.0.10	TCP	338	9000 - 55600	[PSH, ACK]	Seq=1	Ack=281	Win=65536	Len=272	Tsval=2699507935	Tsecr=2156149711			
9	0.003381255	0.000029709	10.10.0.10	10.10.0.11	TCP	66	55600 - 9000	[ACK]	Seq=281	Ack=273	Win=64512	Len=0	Tsval=2156149713	Tsecr=2699507935			
10	0.020869264	0.017488009	10.10.0.10	10.10.0.11	TPKT	70	Continuation										
11	0.001023725	0.000154461	10.10.0.11	10.10.0.10	TCP	66	9000 - 55600	[ACK]	Seq=273	Ack=285	Win=65536	Len=0	Tsval=2699507993	Tsecr=2156149730			
12	0.001065512	0.000041787	10.10.0.10	10.10.0.11	TCP	98	55600 - 9000	[PSH, ACK]	Seq=285	Ack=273	Win=64512	Len=32	Tsval=2156149771	Tsecr=2699507993			
13	0.001092697	0.000027185	10.10.0.11	10.10.0.10	TCP	66	9000 - 55600	[ACK]	Seq=273	Ack=317	Win=65536	Len=0	Tsval=2699507993	Tsecr=2156149771			
14	0.003574711	0.002482014	10.10.0.11	10.10.0.10	TCP	326	9000 - 55600	[PSH, ACK]	Seq=273	Ack=317	Win=65536	Len=260	Tsval=2699507995	Tsecr=2156149771			
15	0.003794953	0.000220242	10.10.0.11	10.10.0.10	TCP	66	9000 - 55600	[FIN, ACK]	Seq=533	Ack=317	Win=65536	Len=0	Tsval=2699507995	Tsecr=2156149771			
16	0.004097171	0.000302218	10.10.0.10	10.10.0.11	TCP	66	55600 - 9000	[FIN, ACK]	Seq=317	Ack=534	Win=64512	Len=0	Tsval=2156149774	Tsecr=2699507995			
17	0.004143301	0.000046130	10.10.0.11	10.10.0.10	TCP	66	9000 - 55600	[ACK]	Seq=534	Ack=318	Win=65536	Len=0	Tsval=2699507996	Tsecr=2156149774			

+ Frame 8: Packet, 330 bytes on wire (2704 bits), 330 bytes captured (2704 bits) on interface labbr, id 0

+ Ethernet II, Src: b2:80:38:be:95:00 (b2:80:38:be:95:00), Dst: 10:00:75:c1:a1:b3 (10:00:75:c1:a1:b3)

+ Internet Protocol Version 4, Src: 10.10.0.11, Dst: 10.10.0.10

+ Transmission Control Protocol, Src Port: 9000, Dst Port: 55600, Seq: 1, Ack: 281, Len: 272

- Data (272 bytes)

Data [..]: 0000010c010200001060000000a400001c2b625e613f827bcafb19a055904a4051fd5943fb521ab99a5e1e0e

[Length: 272]

0000

10 00 75 c1 a1 b3 b2 80

38 be 95 00 00 00 45 00

u 8 0 0 0 E

0010

01 44 01 4a 40 00 40 00

24 42 0a 0a 00 0a 0a 0a

D J0 0' \$B

0020

00 0a 23 29 d9 30 cd 77

fe 0b c2 52 47 36 00 18

(0 ' W ' R6

0030

00 40 15 5f 00 00 01 01

00 0a 00 e7 30 df 00 04

0 0 0 0 0 0 0 0

0040

3b cf 00 00 01 0c 01 02

00 00 10 65 00 00 00 a4

0 0 0 0 0 0 0 0

0050

00 00 1c d2 b6 25 e6 13

f8 27 bc af 01 9a 05 59

0 0 0 0 0 0 0 0

0060

04 a4 05 1f 05 94 3f b5

21 d2 a0 99 a5 e1 ee 80

0 0 0 0 0 0 0 0

0070

c3 17 a8 5e e2 55 e1 15

30 47 7f 19 1a 01 6f fa

0 0 0 0 0 0 0 0

0080

41 b0 7c f5 03 15 3c 2c

39 02 1a 97 e8 7e ec d0

0 0 0 0 0 0 0 0

0090

9e 0a 46 e3 5d 93 41 39

47 50 c1 2c 8b 56 cd 47

0 0 0 0 0 0 0 0

00a0

8b f9 b8 03 d2 6d 4d e1

01 ce 38 b9 e0 ef 9f 00

0 0 0 0 0 0 0 0

00b0

74 be 09 0f bd c2 40 27

2b 3c 40 d2 d8 10 95 4d

0 0 0 0 0 0 0 0

00c0

de 6b 56 b4 ed 59 b3 bf

2b b1 dd e1 fc 42 13 8f

0 0 0 0 0 0 0 0

00d0

cd ab be 79 dd de ff 72

83 e9 32 04 73 b0 03 3b

0 0 0 0 0 0 0 0

00e0

96 b0 b9 54 59 3e 7b d6

b1 65 a8 05 a1 e3 5c b6

0 0 0 0 0 0 0 0

00f0

7b 75 f1 4e 32 da 5d 1c

ef 3d 85 7a f6 e2 bf 1f

0 0 0 0 0 0 0 0

0100

e9 df 00 0a 09 0e 5d 9d

cc f4 04 a4 09 fa 14 6f

0 0 0 0 0 0 0 0

0110

78 0b 87 48 f3 11 dc 1f

65 27 c9 10 50 50 23 8e

x H e' pP#

0120

e8 60 02 6d 53 64 71 f1

14 90 0b 89 d6 5a 05 08

mSdq Ze

0130

5e af 04 19 05 46 04 92

00 e9 ba 25 05 df bd 31

A F 'Se 1

0140

04 4e 6c df db 27 73 79

fa 72 a7 04 a3 d4 d0 0d

Nl 'sy r

0150

3d 45

Рис. 7. Пакет с сеансовым ключом, зашифрованным публичным ключом клиента

1 0.000000000	10.10.0.10	10.10.0.11	TCP	74 55600 → 9000	[SYN] Seq=0 Win=64240 Len=0 MSS=1460 SACK_PERM TSval=2156149710 TSecr=0 WS=1024
2 0.000053546	0.000053546	10.10.0.11	TCP	74 9000 → 55600	[SYN, ACK] Seq=0 Ack=1 Win=65160 Len=0 MSS=1460 SACK_PERM TSval=2699507993 TSecr=2156149710 WS=1024
3 0.000083317	0.000029771	10.10.0.10	TCP	66 55600 → 9000	[ACK] Seq=1 Ack=1 Win=64512 Len=0 TSval=2156149710 TSecr=2699507932
4 0.000097371	0.000014054	10.10.0.10	TPKT	70 Continuation	
5 0.001025216	0.000027845	10.10.0.11	TCP	66 9000 → 55600	[ACK] Seq=1 Ack=5 Win=65536 Len=0 TSval=2699507933 TSecr=2156149711
6 0.001279684	0.000254468	10.10.0.10	TCP	342 55600 → 9000	[PSH, ACK] Seq=5 Ack=1 Win=64512 Len=276 TSval=2156149711 TSecr=2699507933
7 0.001383794	0.000024110	10.10.0.11	TCP	66 9000 → 55600	[ACK] Seq=1 Ack=281 Win=65536 Len=0 TSval=2699507933 TSecr=2156149711
8 0.003351546	0.002047752	10.10.0.11	TCP	338 9000 → 55600	[PSH, ACK] Seq=1 Ack=281 Win=65536 Len=272 TSval=2699507933 TSecr=2156149711
9 0.003381255	0.000029709	10.10.0.10	TCP	66 55600 → 9000	[ACK] Seq=281 Ack=273 Win=64512 Len=0 TSval=2156149713 TSecr=2699507935
10 0.020609264	0.017488009	10.10.0.10	TPKT	70 Continuation	
11 0.061023725	0.040154461	10.10.0.11	TCP	66 9000 → 55600	[ACK] Seq=273 Ack=285 Win=65536 Len=0 TSval=2699507993 TSecr=2156149730
12 0.061065512	0.000041787	10.10.0.10	TCP	98 55600 → 9000	[PSH, ACK] Seq=285 Ack=273 Win=64512 Len=32 TSval=2156149771 TSecr=2699507993
13 0.061092697	0.000027185	10.10.0.11	TCP	66 9000 → 55600	[ACK] Seq=273 Ack=317 Win=65536 Len=0 TSval=2699507993 TSecr=2156149771
14 0.063574711	0.002482014	10.10.0.10	TCP	326 9000 → 55600	[PSH, ACK] Seq=273 Ack=317 Win=65536 Len=260 TSval=2699507995 TSecr=2156149771
15 0.063794953	0.002262242	10.10.0.11	TCP	66 9000 → 55600	[FIN, ACK] Seq=533 Ack=317 Win=65536 Len=0 TSval=2699507995 TSecr=2156149771
16 0.064097171	0.000302218	10.10.0.10	TCP	66 55600 → 9000	[FIN, ACK] Seq=317 Ack=534 Win=64512 Len=0 TSval=2156149774 TSecr=2699507995
17 0.064143301	0.000046130	10.10.0.11	TCP	66 9000 → 55600	[ACK] Seq=534 Ack=318 Win=65536 Len=0 TSval=2699507996 TSecr=2156149774

+ Frame 14: Packet, 326 bytes on wire (2608 bits), 326 bytes captured (2608 bits) on interface labbr, id	0000	16 00 75 c1 a1 b3 b2 80 38 be 95 00 08 00 45 00	0000	00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00	0000	00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
+ Ethernet II, Src: b2:80:38:be:95:60 (b2:80:38:be:95:60), Dst: 16:00:75:c1:a1:b3 (16:00:75:c1:a1:b3)	0010	01 38 01 40 00 00 40 00 24 4b 8a 00 00 0a 0a	0010	00 0a 23 20 09 30 c0 77 ff ab c2 52 47 5a 80 18	0010	00 0a 23 20 09 30 c0 77 ff ab c2 52 47 5a 80 18
+ Internet Protocol Version 4, Src: 10.10.0.11, Dst: 10.10.0.10	0030	00 40 15 53 00 00 01 01 08 0a a0 e7 39 1b 88 84	0030	00 40 15 53 00 00 01 01 08 0a a0 e7 39 1b 88 84	0030	00 40 15 53 00 00 01 01 08 0a a0 e7 39 1b 88 84
+ Transmission Control Protocol, Src Port: 9000, Dst Port: 55600, Seq: 273, Ack: 317, Len: 260	0040	3c 0b 00 00 01 00 1b 10 14 04 49 b0 39 e3 db a1	0040	3c 0b 00 00 01 00 1b 10 14 04 49 b0 39 e3 db a1	0040	3c 0b 00 00 01 00 1b 10 14 04 49 b0 39 e3 db a1
- Data (260 bytes)	0050	b7 09 00 ba 74 b2 5a a5 0b 10 57 46 9b 16 b1 f0	0050	b7 09 00 ba 74 b2 5a a5 0b 10 57 46 9b 16 b1 f0	0050	b7 09 00 ba 74 b2 5a a5 0b 10 57 46 9b 16 b1 f0
Data [...]: 000001001b1b140449b039e3dba1b76909ba74b25aa59b1857469b16b1fbb2c197d9a23da0363a9d080134a15f4	0060	b2 c1 07 d9 a2 3d a0 36 3a 0d 08 01 34 a1 5f 49	0060	b2 c1 07 d9 a2 3d a0 36 3a 0d 08 01 34 a1 5f 49	0060	b2 c1 07 d9 a2 3d a0 36 3a 0d 08 01 34 a1 5f 49
[Length: 260]	0070	90 5c 09 7d bd 07 d7 5a 56 74 44 b9 fb b0 ce d1	0070	90 5c 09 7d bd 07 d7 5a 56 74 44 b9 fb b0 ce d1	0070	90 5c 09 7d bd 07 d7 5a 56 74 44 b9 fb b0 ce d1
	0080	98 22 f9 37 0c 2d 93 17 06 7a 89 22 0b b0 d4	0080	98 22 f9 37 0c 2d 93 17 06 7a 89 22 0b b0 d4	0080	98 22 f9 37 0c 2d 93 17 06 7a 89 22 0b b0 d4
	0090	27 99 cb 68 1b e5 bf 90 89 04 b9 07 c6 ee 5e 0d	0090	27 99 cb 68 1b e5 bf 90 89 04 b9 07 c6 ee 5e 0d	0090	27 99 cb 68 1b e5 bf 90 89 04 b9 07 c6 ee 5e 0d
	00a0	d5 e0 bf ba d9 00 90 3b 57 30 2d ac fb 12 99 a7	00a0	d5 e0 bf ba d9 00 90 3b 57 30 2d ac fb 12 99 a7	00a0	d5 e0 bf ba d9 00 90 3b 57 30 2d ac fb 12 99 a7
	00b0	f1 78 f4 9e bf 98 df 91 4f 58 08 e5 4a 89 ec 4e	00b0	f1 78 f4 9e bf 98 df 91 4f 58 08 e5 4a 89 ec 4e	00b0	f1 78 f4 9e bf 98 df 91 4f 58 08 e5 4a 89 ec 4e
	00c0	91 b3 80 da db 56 ac b5 13 ea a3 1f d9 e4 79 9d	00c0	91 b3 80 da db 56 ac b5 13 ea a3 1f d9 e4 79 9d	00c0	91 b3 80 da db 56 ac b5 13 ea a3 1f d9 e4 79 9d
	00d0	e2 b5 fe f4 7a d0 5c bf 7c 43 9c bd a4 cb 9e fe	00d0	e2 b5 fe f4 7a d0 5c bf 7c 43 9c bd a4 cb 9e fe	00d0	e2 b5 fe f4 7a d0 5c bf 7c 43 9c bd a4 cb 9e fe
	00e0	12 ce ca 00 90 fa 4f 02 0a 29 0f 05 a6 76 fa bc	00e0	12 ce ca 00 90 fa 4f 02 0a 29 0f 05 a6 76 fa bc	00e0	12 ce ca 00 90 fa 4f 02 0a 29 0f 05 a6 76 fa bc
	00f0	03 0d bb 99 0b e3 03 05 1b fa 91 1b 0b 9f b1 2e	00f0	03 0d bb 99 0b e3 03 05 1b fa 91 1b 0b 9f b1 2e	00f0	03 0d bb 99 0b e3 03 05 1b fa 91 1b 0b 9f b1 2e
	0100	d2 8c 24 f0 84 c3 20 f9 2d 1f b8 66 42 0e 44 85	0100	d2 8c 24 f0 84 c3 20 f9 2d 1f b8 66 42 0e 44 85	0100	d2 8c 24 f0 84 c3 20 f9 2d 1f b8 66 42 0e 44 85
	0110	11 ae e8 e9 c3 96 30 e9 03 07 70 83 2a 25 b7 7a	0110	11 ae e8 e9 c3 96 30 e9 03 07 70 83 2a 25 b7 7a	0110	11 ae e8 e9 c3 96 30 e9 03 07 70 83 2a 25 b7 7a
	0120	52 f8 ee 2f e0 42 0f e7 09 04 02 11 ba 07 0e e5	0120	52 f8 ee 2f e0 42 0f e7 09 04 02 11 ba 07 0e e5	0120	52 f8 ee 2f e0 42 0f e7 09 04 02 11 ba 07 0e e5
	0130	f8 78 48 c4 92 ed 1a 3d 7d 38 2e c1 6e ce 52 b5	0130	f8 78 48 c4 92 ed 1a 3d 7d 38 2e c1 6e ce 52 b5	0130	f8 78 48 c4 92 ed 1a 3d 7d 38 2e c1 6e ce 52 b5
	0140	56 9b ba bf 03 da	0140	56 9b ba bf 03 da	0140	56 9b ba bf 03 da

Рис. 8. Зашифрованные запрос и ответ (обмен данными, отсутствие читаемого текста)

3. Разработка сервера

3.1. Структура программы

Сервер реализован как консольное приложение на языке C с использованием WinSock2, портов завершения ввода-вывода и CryptoAPI. Программа не взаимодействует с пользователем, вся информация о работе выводится на консоль. Код разделён на модули.

Файл	Назначение
server.c	Основной модуль сервера: порт завершения, приём подключений, рабочие потоки, конечный автомат подключения, диагностический вывод
sysinfo.c	Разбор запросов и получение информации о системе (ОС, время, память, диски, права доступа, владелец)
common.c, common.h	Общий для клиента и сервера код: формирование и разбор записей протокола, кадры, функции CryptoAPI, работа с Юникодом и консолью

Таблица 8. Модули сервера

3.2. Параллельная обработка запросов на портах завершения

При запуске сервер инициализирует WinSock (WSAStartup), криптопровайдер и создаёт порт завершения функцией CreateIoCompletionPort. Слушающий сокет создаётся функцией WSASocket с флагом WSA_FLAG_OVERLAPPED, привязывается к порту и переводится в режим прослушивания. Порт по умолчанию 9000, его можно задать параметром командной строки.

Подключения принимаются асинхронно функцией AcceptEx. Так как она не создаёт сокет сама, для каждой операции заранее создаётся сокет, а в область данных принимается пара адресов (буфер на 16 байт больше размера sockaddr_in на каждый адрес, как требует документация). Одновременно запущено 8 операций AcceptEx: после завершения каждой из них сразу запускается новая, поэтому сервер всегда готов принять клиента. После при-

нения подключения вызывается GetAcceptExSockaddrs, из которого берётся адрес клиента для диагностического вывода.

Обработка завершённых операций выполняется пулом рабочих потоков. Число потоков равно удвоенному числу процессоров, все потоки вызывают GetQueuedCompletionStatus на одном порту, и система сама передаёт каждое уведомление свободному потоку. Тем самым запросы разных клиентов обрабатываются параллельно. Структура OVERLAPPED вложена в служебную структуру io_or, которая хранит тип операции (приём подключения, чтение, запись). Получив указатель на OVERLAPPED, поток по макросу CONTAINING_RECORD восстанавливает io_or и определяет, какая операция завершилась. В качестве ключа порта для подключения клиента используется указатель на его контекст (структуру conn), поэтому поиск по массиву не нужен. Контекст содержит сокет, состояние, буферы приёма и отправки, счётчики принятых и отправленных байт, сеансовый ключ и время последней активности.

Жизненный цикл подключения описывается конечным автоматом, состояние которого хранится в контексте клиента.

Состояние	Что ожидается или выполняется	Следующее состояние
ST_WAIT_PUBKEY	Приём кадра с публичным ключом клиента, создание сеансового ключа	ST_SEND_KEY
ST_SEND_KEY	Отправка сеансового ключа, зашифрованного публичным ключом клиента	ST_WAIT_REQUEST
ST_WAIT_REQUEST	Приём зашифрованного запроса, расшифровка, обработка, шифрование ответа	ST_SEND_RESPONSE
ST_SEND_RESPONSE	Отправка зашифрованного ответа, затем закрытие подключения	подключение закрыто

Таблица 9. Состояния подключения

Данные могут приходить и уходить частями, поэтому при чтении сервер накапливает байты в буфере и ждёт, пока кадр придёт целиком (заголовок содержит его длину), а при отправке, если система передала меньше запрошенного, досылает остаток. Для каждого подключения в любой момент времени ожидает завершения ровно одна операция: после приёма запускается отправка, после отправки следующий приём или закрытие. Благодаря этому в обработчике завершения операции сокет и контекст можно безопасно закрыть и освободить, а механизм отмены операций через `CancelIo` с событием от `PostQueuedCompletionStatus`, приведённый в примере методических указаний и нужный при нескольких одновременных операциях на сокете, не потребовался. Буферы всегда расположены в контексте клиента, который не освобождается, пока операция не завершена.

Дополнительно реализованы следующие меры надёжности. Бездействующие подключения закрываются через 30 секунд: главный поток раз в секунду просматривает список подключений и вызывает `CancelIoEx`, после чего операция завершается с ошибкой и подключение закрывается обычным путём. Число одновременных подключений ограничено (2000), размер принимаемого кадра ограничен 16 КБ. Остановка сервера выполняется по `Ctrl+C`: слушающий сокет закрывается, а в порт отправляется по одному служебному уведомлению на каждый рабочий поток.

3.3. Шифрование на стороне сервера

Получив первый кадр, сервер считает его публичным ключом клиента. Функция `crypto_create_session` импортирует ключ (`CryptImportKey`), генерирует сеансовый ключ AES-256 (`CryptGenKey` с `CALG_AES_256`) и экспортирует его в формате `SIMPLEBLOB`, зашифрованным публичным ключом клиента (`CryptExportKey`). Полученный блок отправляется клиенту, а дескриптор сеансового ключа сохраняется в контексте подключения и освобождается (`CryptDestroyKey`) при его закрытии. Все последующие сообщения расшифровываются функцией `crypto_decrypt`, а ответ шифруется функцией `crypto_encrypt`, которая перед шифртекстом помещает случайный вектор инициализации. Так как функции `CryptoAPI` вызываются из нескольких потоков, обращения к ним защищены критической секцией.

3.4. Получение информации о системе

Запрос разбирается функцией `process_request`, которая по имени команды вызывает соответствующий обработчик. Результат формируется в виде записей протокола. Используемые функции WinAPI приведены в таблице.

Запрос	Функции WinAPI	Особенности
GET_OS	RtlGetVersion, GetNativeSystemInfo, GetComputerName	Версия ОС определяется функцией <code>RtlGetVersion</code> из <code>ntdll.dll</code> : <code>GetVersionEx</code> без манифеста возвращает неверную версию в Windows 8.1 и выше. Название продукта берётся из реестра, а по номеру версии и типу продукта определяется название ОС
GET_TIME	GetSystemTime, GetLocalTime, GetTimeZoneInformation	Передаются время UTC и местное время по компонентам, день недели, время в формате Unix, смещение и название часового пояса
GET_UPTIME	GetTickCount64	Возвращается число миллисекунд с запуска и вычисленное время запуска ОС. 64-битная функция не переполняется через 49,7 суток
GET_MEMORY	GlobalMemoryStatusEx	Процент загрузки, физическая память, файл подкачки, виртуальное адресное пространство, всего и свободно. Поля 64-битные, что поз-

		воляет работать с объёмом больше 4 ГБ
GET_DRIVES	GetLogicalDriveStrings, GetDriveType, GetVolumeInformation	Для каждого диска возвращаются тип, класс (локальный, сетевой, съёмный), файловая система, метка тома и признак готовности
GET_FREESPACE	GetLogicalDriveStrings, GetDriveType, GetDiskFreeSpaceEx	Рассматриваются локальные диски: фиксированные и съёмные с носителем (USB-накопители). Сетевые диски и приводы компакт-дисков пропускаются
GET_ACL	GetNamedSecurityInfo, GetSecurityDescriptorControl, GetAclInformation, GetAce, LookupAccountSid, ConvertSidToStringSid	Описано ниже
GET_OWNER	GetNamedSecurityInfo, LookupAccountSid, ConvertSidToStringSid	Описано ниже

Таблица 10. Функции WinAPI, используемые для получения информации

Типы дисков, возвращаемые функцией `GetDriveType`, сопоставляются с классами следующим образом: `DRIVE_FIXED` и `DRIVE_RAMDISK` это локальные диски, `DRIVE_REMOTE` это сетевой диск, `DRIVE_REMOVABLE` и `DRIVE_CDROM` это съёмные носители. Файловая система (например, NTFS или FAT32) определяется функцией `GetVolumeInformation`. Если носителя нет, в ответе помечается, что диск не готов.

3.4.1. Права доступа и владелец

Для получения прав доступа к объекту используется функция `GetNamedSecurityInfo`. Объект задаётся типом (`FILE` для файла или папки,

REG для ключа реестра) и путём. Для файловых объектов используется тип SE_FILE_OBJECT, а для ключей реестра SE_REGISTRY_KEY; при этом корень реестра в пути, заданный в привычной форме (HKLM, HKCU, HKCR, HKU или полное имя вида HKEY_LOCAL_MACHINE), преобразуется к форме, которую понимает функция (MACHINE, CURRENT_USER, CLASSES_ROOT, USERS). Запрашивается информация DACL_SECURITY_INFORMATION, из которой функции GetAclInformation и GetAce извлекают элементы списка контроля доступа (ACE).

Для каждого элемента определяются:

- SID субъекта: его положение в элементе зависит от типа ACE (для разрешающих и запрещающих записей сразу после маски, для объектных записей после дополнительных полей), по SID функцией LookupAccountSid определяются имя субъекта и его тип, а функцией ConvertSidToStringSid строковое представление;
- тип ACE (разрешающий, запрещающий, аудит и другие);
- флаги ACE и область действия записи, вычисляемая по флагам наследования;
- маска доступа, а также номера установленных в ней битов и их названия для текущего типа объекта.

Названия битов определяются по таблице констант, отличающейся для файла, папки и ключа реестра: например, бит 0 это FILE_READ_DATA для файла, FILE_LIST_DIRECTORY для папки и KEY_QUERY_VALUE для ключа реестра. Стандартные права (биты 16–20) и универсальные права (биты 28–31) одинаковы для всех типов.

Бит	Файл	Папка	Ключ реестра
0	FILE_READ_DATA	FILE_LIST_DIRECTORY	KEY_QUERY_VALUE
1	FILE_WRITE_DATA	FILE_ADD_FILE	KEY_SET_VALUE
2	FILE_APPEND_DATA	FILE_ADD_SUBDIRECTORY	KEY_CREATE_SUB_KEY
3	FILE_READ_EA	FILE_READ_EA	KEY_ENUMERATE_SUB_KEYS
4	FILE_WRITE_EA	FILE_WRITE_EA	KEY_NOTIFY
5	FILE_EXECUTE	FILE_TRAVERSE	KEY_CREATE_LINK
6	FILE_DELETE_CHILD	FILE_DELETE_CHILD	не используется
7	FILE_READ_ATTRIBUTES	FILE_READ_ATTRIBUTES	не используется

8	FILE_WRITE_ATTRIBUTES	FILE_WRITE_ATTRIBUTES	Не используется
16	DELETE	DELETE	DELETE
17	READ_CONTROL	READ_CONTROL	READ_CONTROL
18	WRITE_DAC	WRITE_DAC	WRITE_DAC
19	WRITE_OWNER	WRITE_OWNER	WRITE_OWNER
20	SYNCHRONIZE	SYNCHRONIZE	SYNCHRONIZE
24	ACCESS_SYSTEM_SECURITY	ACCESS_SYSTEM_SECURITY	ACCESS_SYSTEM_SECURITY
25	MAXIMUM_ALLOWED	MAXIMUM_ALLOWED	MAXIMUM_ALLOWED
28–31	GENERIC_ALL, GENERIC_EXECUTE, GENERIC_WRITE, GENERIC_READ	то же	то же

Таблица 11. Названия битов маски доступа для различных типов объектов

Область действия записи определяется комбинацией флагов наследования ACE: OBJECT_INHERIT_ACE (наследуется файлами), CONTAINER_INHERIT_ACE (наследуется вложенными папками или подразделами), INHERIT_ONLY_ACE (запись на сам объект не действует, а только передаётся вниз), NO_PROPAGATE_INHERIT_ACE (наследуется только на один уровень) и INHERITED_ACE (запись получена от родителя). Например, запись с флагами OBJECT_INHERIT_ACE и CONTAINER_INHERIT_ACE действует на эту папку, вложенные папки и файлы, а запись с флагами CONTAINER_INHERIT_ACE и INHERIT_ONLY_ACE только на вложенные папки. Дополнительно сервер сообщает, отключено ли наследование от родителя (признак защищённого DACL), и обрабатывает особый случай NULL DACL, при котором доступ разрешён всем без ограничений.

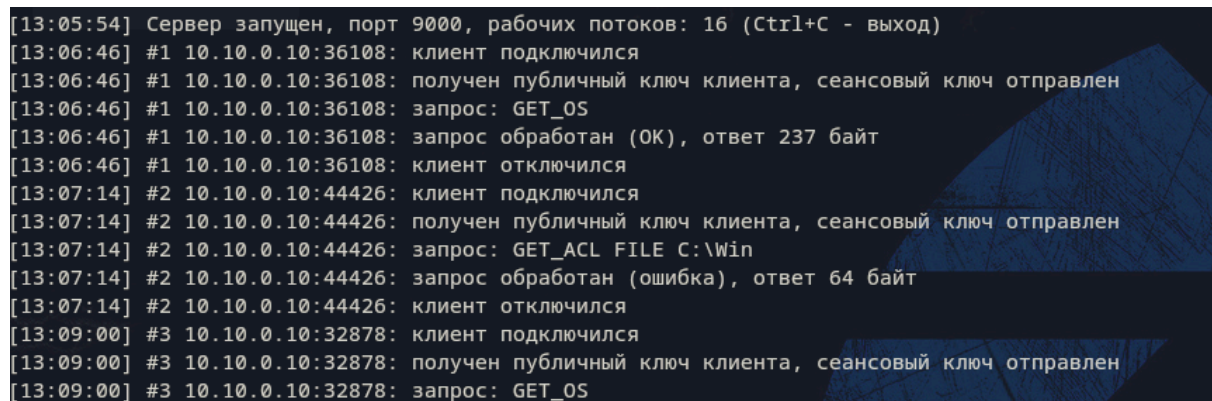
Владелец объекта определяется тем же вызовом GetNamedSecurityInfo, но с информацией OWNER_SECURITY_INFORMATION. В ответ включаются SID владельца, его имя в виде домен\имя и тип учётной записи. Если системная функция завершилась ошибкой (например, нет доступа к объекту или путь не существует), клиенту передаётся код ошибки и текст, полученный функцией FormatMessage.

3.5. Диагностический вывод

В соответствии с заданием сервер выводит на консоль диагностическую информацию. Каждая строка начинается со времени и содержит номер подключения и адрес клиента:

[ЧЧ:ММ:СС] #номер IP-адрес:порт: событие

Выводятся следующие события: запуск сервера (порт и число рабочих потоков), подключение клиента, получение публичного ключа и отправка сеансового ключа, принятый запрос (с аргументами), результат обработки (успех или ошибка, размер ответа), отключение клиента, а также ошибки (неверная длина кадра, ошибка расшифровки, ошибки ввода-вывода, превышение времени бездействия, достижение предела числа подключений) и остановка сервера. Строка формируется целиком и выводится одним вызовом под защитой критической секции, поэтому сообщения разных потоков не перемешиваются. Вывод выполняется в Юникоде, и кириллица в сообщениях и путях отображается корректно.



```
[13:05:54] Сервер запущен, порт 9000, рабочих потоков: 16 (Ctrl+C - выход)
[13:06:46] #1 10.10.0.10:36108: клиент подключился
[13:06:46] #1 10.10.0.10:36108: получен публичный ключ клиента, сеансовый ключ отправлен
[13:06:46] #1 10.10.0.10:36108: запрос: GET_OS
[13:06:46] #1 10.10.0.10:36108: запрос обработан (OK), ответ 237 байт
[13:06:46] #1 10.10.0.10:36108: клиент отключился
[13:07:14] #2 10.10.0.10:44426: клиент подключился
[13:07:14] #2 10.10.0.10:44426: получен публичный ключ клиента, сеансовый ключ отправлен
[13:07:14] #2 10.10.0.10:44426: запрос: GET_ACL FILE C:\Win
[13:07:14] #2 10.10.0.10:44426: запрос обработан (ошибка), ответ 64 байт
[13:07:14] #2 10.10.0.10:44426: клиент отключился
[13:09:00] #3 10.10.0.10:32878: клиент подключился
[13:09:00] #3 10.10.0.10:32878: получен публичный ключ клиента, сеансовый ключ отправлен
[13:09:00] #3 10.10.0.10:32878: запрос: GET_OS
```

Рис. 9. Журнал сервера при обработке запроса одного клиента

4. Разработка клиента

4.1. Пользовательский интерфейс

Клиент представляет собой консольное приложение. После запуска он однократно генерирует пару ключей RSA-2048, а затем запрашивает адрес сервера (имя или IP-адрес, при необходимости с портом через двоеточие). Далее выводится меню, в котором пользователь выбирает тип запроса. Выбор пункта сразу инициирует запрос: клиент подключается к серверу, выполняет обмен ключами, отправляет запрос и выводит полученную информацию. Для запросов прав доступа и владельца дополнительно вводятся тип объекта (файл или папка, либо ключ реестра) и путь.

Пункт	Действие	Запрос	Дополнительный ввод
1	Тип и версия ОС	GET_OS	нет
2	Текущее время	GET_TIME	нет
3	Время с момента запуска ОС	GET_UPTIME	нет
4	Информация о памяти	GET_MEMORY	нет
5	Типы подключённых дисков и файловые системы	GET_DRIVES	нет
6	Свободное место на локальных дисках	GET_FREESPACE	нет
7	Права доступа к файлу, папке или ключу реестра	GET_ACL	тип объекта и путь
8	Владелец файла, папки или ключа реестра	GET_OWNER	тип объекта и путь
s	Смена адреса сервера	—	адрес сервера
r	Включение и выключение вывода сырого ответа	—	нет

0	Выход	—	нет
---	-------	---	-----

Таблица 12. Пункты меню клиента

```

Генерация пары ключей RSA-2048...
Адрес сервера (IP или имя, можно host:порт) [127.0.0.1]:
the environment.
10.10.0.11

=== Сервер: 10.10.0.11:9000 ===
1 Тип и версия ОС
2 Текущее время
3 Время с момента запуска ОС
4 Информация о памяти
5 Типы подключённых дисков и файловые системы
6 Свободное место на локальных дисках
7 Права доступа к файлу/папке/ключу реестра
8 Владелец файла/папки/ключа реестра
s Сменить адрес сервера
r Вкл/выкл вывод сырого ответа
0 Выход
Выбор: 1

Операционная система
Имя компьютера : ARCHLINUX
ОС              : Windows 10
Product name    : Windows 10 Pro
Версия выпуска  : 22H2
Версия          : 10.0, сборка 19045
Service Pack    : нет (0.0)
Тип             : рабочая станция
Архитектура     : x64

```

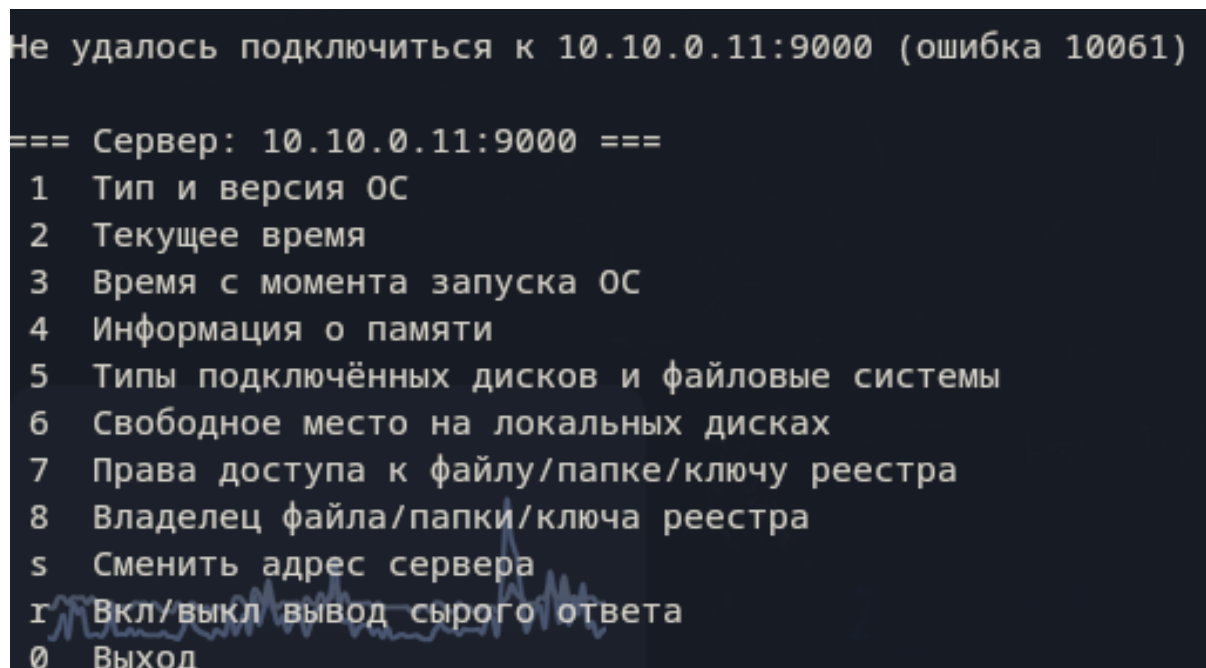
Рис. 10. Главное меню клиента и пример работы

4.2. Выполнение запроса

Для каждого запроса клиент выполняет следующие действия:

- открывает подключение к серверу (имя разрешается функцией `getaddrinfo`, на ожидание ответа установлен таймаут 30 секунд);
- отправляет серверу свой публичный ключ и получает сеансовый ключ, который расшифровывает приватным ключом (`CryptImportKey`);
- шифрует запрос сеансовым ключом и отправляет его;
- принимает зашифрованный ответ, расшифровывает его и разбирает на записи;
- закрывает подключение и освобождает ключи.

Если сервер недоступен, не прислал ключ или ответ, а также если сообщение не удалось расшифровать, клиент выводит соответствующее сообщение, а программа остаётся работоспособной и ожидает следующей команды. Если сервер вернул ошибку, выводятся её код, источник (протокол или система) и текст.



```
Не удалось подключиться к 10.10.0.11:9000 (ошибка 10061)

=== Сервер: 10.10.0.11:9000 ===
1 Тип и версия ОС
2 Текущее время
3 Время с момента запуска ОС
4 Информация о памяти
5 Типы подключённых дисков и файловые системы
6 Свободное место на локальных дисках
7 Права доступа к файлу/папке/ключу реестра
8 Владелец файла/папки/ключа реестра
s Сменить адрес сервера
r Вкл/выкл вывод сырого ответа
0 Выход
```

Рис. 11. Сообщение об ошибке при недоступном сервере

4.3. Вывод полученной информации

Полученный ответ разбирается на записи, после чего значения преобразуются к виду, удобному для чтения. Числа байт переводятся в килобайты, мегабайты и гигабайты с одновременным выводом точного числа байт. Класс диска (локальный, сетевой, съёмный) выводится по-русски, а тип по названию, возвращённому сервером. Для запроса GET_OS выводятся имя компьютера, название и версия ОС, пакет обновления, тип (рабочая станция или сервер) и разрядность.

Текущее время выводится в виде даты, времени с миллисекундами и дня недели, отдельно приводятся значения года, месяца, дня, часов, минут и секунд, а также часовой пояс. Время, прошедшее с момента запуска ОС, выводится в днях, часах, минутах, секундах и миллисекундах, а также в суммарном виде в секундах, минутах, часах и днях.

Права доступа выводятся по каждой записи ACE и включают все элементы, требуемые заданием: SID субъекта, имя субъекта, тип ACE, область действия установленных прав, номера установленных битов маски доступа и названия этих битов для текущего типа объекта (названия констант по MSDN). Дополнительно выводятся флаги ACE, маска в шестнадцатеричном виде и признак наследования. Для владельца выводятся SID и имя владельца.

В режиме вывода сырого ответа (пункт меню r) перед отформатированным результатом выводится ответ сервера в исходном виде, что удобно для проверки протокола.

5. Выводы

Были получены навыки организации параллельной обработки большого количества клиентских соединений при помощи механизма портов завершения Win32. Были получены навыки реализации шифрования передаваемых данных при помощи средств CryptoAPI. Были получены навыки извлечения информации о системе.

Приложения

Приложение А

server.c

```
#include "common.h"
#include <mswsock.h>
#include <process.h>

#define ACCEPT_POSTED 8
#define MAX_CONNS 2000
#define IDLE_TIMEOUT_MS 30000
#define KEY_LISTEN ((ULONG_PTR)1)
#define KEY_STOP ((ULONG_PTR)2)

typedef enum
{
    OP_ACCEPT = 1,
    OP_RECV,
    OP_SEND
} op_type;

typedef struct
{
    OVERLAPPED ov; /* должен быть первым */
    op_type type;
} io_op;

typedef struct accept_ctx
{
    io_op op;
    SOCKET sock;
    DWORD bytes;
    char addrs[2 * (sizeof(struct sockaddr_in) + 16)];
} accept_ctx;

typedef enum
{
    ST_WAIT_PUBKEY,
```

```

    ST_SEND_KEY,
    ST_WAIT_REQUEST,
    ST_SEND_RESPONSE
} conn_state;

typedef struct conn
{
    io_op rx_op, tx_op;
    SOCKET sock;
    conn_state state;
    unsigned id;
    char peer[40];
    DWORD rx_flags;
    DWORD rx_len;
    BYTE rx[4 + SERVER_RX_MAX];
    BYTE* tx;
    DWORD tx_len, tx_sent;
    HCRYPTKEY sess;
    volatile DWORD last_tick;
    struct conn *prev, *next;
} conn;

static HANDLE g_port;
static SOCKET g_listen = INVALID_SOCKET;
static volatile LONG g_stop;
static volatile LONG g_next_id;
static conn* g_head;
static int g_nconns;
static CRITICAL_SECTION g_list_cs;

/*
-----
*/

static void log_line(const char* fmt, ...)
{
    SYSTEMTIME t;
    char head[32];
    va_list ap, ap2;
    GetLocalTime(&t);
    sprintf(head, "[%02u:%02u:%02u] ", t.wHour, t.wMinute,

```

```

t.wSecond);

va_start(ap, fmt);
va_copy(ap2, ap);
int n = _vscprintf(fmt, ap);
char* body = (char*)malloc((size_t)n + 1);
if (body)
{
    _vsnprintf(body, (size_t)n + 1, fmt, ap2);
    body[n] = 0;
    /* одним вызовом, чтобы строки разных потоков не
перемешивались */
    char* line = (char*)malloc(strlen(head) + (size_t)n + 2);
    if (line)
    {
        sprintf(line, "%s%s\n", head, body);
        con_write(line);
        free(line);
    }
    free(body);
}
va_end(ap2);
va_end(ap);
}

/*
-----
*/
/* Подключения
*/
/*
-----
*/
static conn* conn_new(SOCKET s, const struct sockaddr_in* peer)
{
    conn* c;
    EnterCriticalSection(&g_list_cs);
    if (g_nconns >= MAX_CONNS)
    {
        LeaveCriticalSection(&g_list_cs);
    }
}

```

```

        return NULL;
    }
    c = (conn*)calloc(1, sizeof(conn));
    if (!c)
    {
        LeaveCriticalSection(&g_list_cs);
        return NULL;
    }
    c->sock = s;
    c->id = (unsigned)InterlockedIncrement(&g_next_id);
    c->last_tick = GetTickCount();
    if (peer)
    {
        unsigned ip = ntohl(peer->sin_addr.s_addr);
        sprintf(c->peer, "%u.%u.%u.%u", (ip >> 24) & 255, (ip >>
16) & 255,
            (ip >> 8) & 255, ip & 255, ntohs(peer->sin_port));
    }
    else
    {
        strcpy(c->peer, "?");
    }
    c->next = g_head;
    if (g_head)
        g_head->prev = c;
    g_head = c;
    g_nconns++;
    LeaveCriticalSection(&g_list_cs);
    return c;
}

```

/* Закрыть подключение. Вызывать только когда у сокета нет незавершённых

* операций. */

```

static void conn_close(conn* c)
{
    EnterCriticalSection(&g_list_cs);
    if (c->prev)
        c->prev->next = c->next;
    else

```

```

    g_head = c->next;
    if (c->next)
        c->next->prev = c->prev;
    g_nconns--;
    LeaveCriticalSection(&g_list_cs);

    closesocket(c->sock);
    crypto_free_key(c->sess);
    free(c->tx);
    log_line("#%u %s: клиент отключился", c->id, c->peer);
    free(c);
}

```

/* Запуск асинхронных операций. Возвращают FALSE, если подключение уже закрыто.

```

*/
static BOOL post_recv(conn* c)
{
    WSABUF b;
    b.buf = (char*)c->rx + c->rx_len;
    b.len = (ULONG)(sizeof(c->rx) - c->rx_len);
    memset(&c->rx_op.ov, 0, sizeof(OVERLAPPED));
    c->rx_op.type = OP_RECV;
    c->rx_flags = 0;
    if (WSARecv(c->sock, &b, 1, NULL, &c->rx_flags, &c->rx_op.ov,
    NULL) ==
        SOCKET_ERROR &&
        WSAGetLastError() != WSA_IO_PENDING)
    {
        conn_close(c);
        return FALSE;
    }
    return TRUE;
}

```

```

static BOOL post_send(conn* c)
{
    WSABUF b;
    b.buf = (char*)c->tx + c->tx_sent;
    b.len = c->tx_len - c->tx_sent;
}

```

```

    memset(&c->tx_op.ov, 0, sizeof(OVERLAPPED));
    c->tx_op.type = OP_SEND;
    if (WSASend(c->sock, &b, 1, NULL, 0, &c->tx_op.ov, NULL) ==
SOCKET_ERROR &&
        WSAGetLastError() != WSA_IO_PENDING)
    {
        conn_close(c);
        return FALSE;
    }
    return TRUE;
}

```

/* Поставить кадр на отправку (забирает владение buf, buf - тело без заголовка)

```

*/
static BOOL start_send_frame(conn* c, BYTE* body, DWORD len,
conn_state next)
{
    DWORD flen;
    BYTE* f = frame_wrap(body, len, &flen);
    free(body);
    if (!f)
    {
        conn_close(c);
        return FALSE;
    }
    free(c->tx);
    c->tx = f;
    c->tx_len = flen;
    c->tx_sent = 0;
    c->state = next;
    c->rx_len = 0;
    return post_send(c);
}

```

```

static void on_recv(conn* c, DWORD n)
{
    if (n == 0)
    { /* клиент закрыл соединение */
        conn_close(c);
    }
}

```

```

    return;
}
c->rx_len += n;
if (c->rx_len < 4)
{
    post_recv(c);
    return;
}
DWORD flen = frame_be32(c->rx);
if (flen == 0 || flen > SERVER_RX_MAX)
{
    log_line("#%u %s: некорректная длина кадра (%lu), соединение
закрыто",
        c->id, c->peer, (unsigned long)flen);
    conn_close(c);
    return;
}
if (c->rx_len < 4 + flen)
{ /* кадр пришёл не целиком */
    post_recv(c);
    return;
}

const BYTE* body = c->rx + 4;

if (c->state == ST_WAIT_PUBKEY)
{
    BYTE* blob = NULL;
    DWORD blen = 0;
    if (!crypto_create_session(body, flen, &c->sess, &blob,
&blen))
    {
        log_line("#%u %s: ошибка обмена ключами, соединение
закрыто", c->id,
            c->peer);
        conn_close(c);
        return;
    }
    log_line(
        "%u %s: получен публичный ключ клиента, сеансовый ключ

```

```

отправлен",
    c->id, c->peer);
    start_send_frame(c, blob, blen, ST_SEND_KEY);
}
else if (c->state == ST_WAIT_REQUEST)
{
    DWORD plen = 0;
    BYTE* plain = crypto_decrypt(c->sess, body, flen, &plen);
    if (!plain)
    {
        log_line(
            "#%u %s: не удалось расшифровать запрос, соединение
закрито",
            c->id, c->peer);
        conn_close(c);
        return;
    }
    char summary[1024];
    request_summary((const char*)plain, summary, sizeof(summary));
    log_line("#%u %s: запрос: %s", c->id, c->peer, summary);

    sbuf out;
    sb_init(&out);
    int ok = process_request((const char*)plain, &out);
    free(plain);
    log_line("#%u %s: запрос обработан (%s), ответ %lu байт", c-
>id,
        c->peer, ok ? "OK" : "ошибка", (unsigned long)out.len);

    DWORD elen = 0;
    BYTE* enc = crypto_encrypt(c->sess, (const BYTE*)out.data,
        (DWORD)out.len, &elen);
    sb_free(&out);
    if (!enc)
    {
        conn_close(c);
        return;
    }
    start_send_frame(c, enc, elen, ST_SEND_RESPONSE);
}

```

```

else
{
    conn_close(c);
}
}

static void on_send(conn* c, DWORD n)
{
    c->tx_sent += n;
    if (n == 0)
    {
        conn_close(c);
        return;
    }
    if (c->tx_sent < c->tx_len)
    { /* отправлено не всё - досылаем остаток */
        post_send(c);
        return;
    }
    free(c->tx);
    c->tx = NULL;
    if (c->state == ST_SEND_KEY)
    {
        c->state = ST_WAIT_REQUEST;
        c->rx_len = 0;
        post_recv(c);
    }
    else
    {
        conn_close(c); /* ответ отправлен полностью */
    }
}

/*
-----
*/
/* Приём подключений
*/
/*
-----

```

```

*/
static void post_accept(accept_ctx* a)
{
    a->sock = WSASocketW(AF_INET, SOCK_STREAM, IPPROTO_TCP, NULL, 0,
        WSA_FLAG_OVERLAPPED);
    if (a->sock == INVALID_SOCKET)
    {
        log_line("WSASocket: ошибка %d", WSAGetLastError());
        free(a);
        return;
    }
    memset(&a->op.ov, 0, sizeof(OVERLAPPED));
    a->op.type = OP_ACCEPT;
    a->bytes = 0;
    if (!AcceptEx(g_listen, a->sock, a->addrs, 0,
        sizeof(struct sockaddr_in) + 16,
        sizeof(struct sockaddr_in) + 16, &a->bytes, &a->op.ov)
    &&
        WSAGetLastError() != WSA_IO_PENDING)
    {
        log_line("AcceptEx: ошибка %d", WSAGetLastError());
        closesocket(a->sock);
        free(a);
    }
}

static void on_accept(accept_ctx* a, BOOL ok)
{
    SOCKET s = a->sock;
    if (ok)
    {
        struct sockaddr *local = NULL, *remote = NULL;
        int ll = 0, rl = 0;
        setsockopt(s, SOL_SOCKET, SO_UPDATE_ACCEPT_CONTEXT,
            (char*)&g_listen,
                sizeof(g_listen));
        GetAcceptExSockaddrs(a->addrs, 0, sizeof(struct sockaddr_in) +
16,
            sizeof(struct sockaddr_in) + 16, &local, &ll,
            &remote, &rl);
    }
}

```

```

conn* c = conn_new(s, (const struct sockaddr_in*)remote);
if (!c)
{
    log_line("достигнут лимит подключений, клиент отклонён");
    closesocket(s);
}
else if (CreateIoCompletionPort((HANDLE)s, g_port,
(ULONG_PTR)c, 0) ==
        NULL)
{
    log_line("CreateIoCompletionPort: ошибка %lu",
        (unsigned long)GetLastError());
    conn_close(c);
}
else
{
    log_line("#%u %s: клиент подключился", c->id, c->peer);
    c->state = ST_WAIT_PUBKEY;
    post_recv(c);
}
}
else
{
    closesocket(s);
}
if (g_stop)
{
    free(a);
}
else
{
    post_accept(a); /* сразу готовимся к следующему клиенту */
}
}

/*
-----
*/
/* Рабочий поток
*/

```

```

/*
-----
*/
static unsigned __stdcall worker(void* arg)
{
    (void)arg;
    for (;;)
    {
        DWORD n = 0;
        ULONG_PTR key = 0;
        OVERLAPPED* ov = NULL;
        BOOL ok = GetQueuedCompletionStatus(g_port, &n, &key, &ov,
INFINITE);
        DWORD err = ok ? 0 : GetLastError();

        if (!ov)
        {
            if (key == KEY_STOP || !ok)
                break;
            continue;
        }
        io_op* op = CONTAINING_RECORD(ov, io_op, ov);
        if (op->type == OP_ACCEPT)
        {
            on_accept(CONTAINING_RECORD(op, accept_ctx, op), ok);
            continue;
        }

        conn* c = (conn*)key;
        c->last_tick = GetTickCount();
        if (!ok)
        {
            /* сброс соединения, отмена по таймауту и т.п. */
            if (err != ERROR_OPERATION_ABORTED && err !=
ERROR_NETNAME_DELETED)
                log_line("#%u %s: ошибка ввода-вывода %lu", c->id, c-
>peer,
                        (unsigned long)err);
            conn_close(c);
        }
        else if (op->type == OP_RECV)

```

```

    {
        on_recv(c, n);
    }
    else
    {
        on_send(c, n);
    }
}
return 0;
}

/* Закрытие бездействующих подключений */
static void watchdog(void)
{
    DWORD now = GetTickCount();
    EnterCriticalSection(&g_list_cs);
    for (conn* c = g_head; c; c = c->next)
    {
        if (now - c->last_tick > IDLE_TIMEOUT_MS)
        {
            log_line("#%u %s: таймаут бездействия", c->id, c->peer);
            c->last_tick = now; /* не спамить в журнал */
            CancelIoEx((HANDLE)c->sock,
                NULL); /* незавершённая операция вернётся с ошибкой
*/
        }
    }
    LeaveCriticalSection(&g_list_cs);
}

static BOOL WINAPI ctrl_handler(DWORD type)
{
    (void)type;
    InterlockedExchange(&g_stop, 1);
    return TRUE;
}

int main(int argc, char** argv)
{
    WSADATA wsa;

```

```

unsigned short port = PROTO_PORT;
HANDLE threads[64];
int nthreads;

con_init();
if (argc > 1)
    port = (unsigned short)atoi(argv[1]);

SetErrorMode(SEM_FAILCRITICALERRORS); /* без окон "вставьте
диск" при опросе
                                дисков */
InitializeCriticalSection(&g_list_cs);

if (WSAStartup(MAKEWORD(2, 2), &wsa) != 0)
{
    log_line("WSAStartup: ошибка");
    return 1;
}
if (!crypto_init())
    return 1;

g_port = CreateIoCompletionPort(INVALID_HANDLE_VALUE, NULL, 0,
0);
if (!g_port)
{
    log_line("CreateIoCompletionPort: ошибка %lu",
            (unsigned long)GetLastError());
    return 1;
}

g_listen = WSASocketW(AF_INET, SOCK_STREAM, IPPROTO_TCP, NULL,
0,
                        WSA_FLAG_OVERLAPPED);
struct sockaddr_in addr;
memset(&addr, 0, sizeof(addr));
addr.sin_family = AF_INET;
addr.sin_addr.s_addr = htonl(INADDR_ANY);
addr.sin_port = htons(port);
if (g_listen == INVALID_SOCKET ||
    bind(g_listen, (struct sockaddr*)&addr, sizeof(addr)) != 0 ||

```

```

    listen(g_listen, SOMAXCONN) != 0)
{
    log_line("не удалось занять порт %u (ошибка %d)", port,
        WSAGetLastError());
    return 1;
}
if (!CreateIoCompletionPort((HANDLE)g_listen, g_port,
KEY_LISTEN, 0))
{
    log_line("CreateIoCompletionPort(listen): ошибка %lu",
        (unsigned long)GetLastError());
    return 1;
}

SYSTEM_INFO si;
GetSystemInfo(&si);
nthreads = (int)si.dwNumberOfProcessors * 2;
if (nthreads < 2)
    nthreads = 2;
if (nthreads > 64)
    nthreads = 64;
for (int i = 0; i < nthreads; i++)
    threads[i] = (HANDLE)_beginthreadex(NULL, 0, worker, NULL, 0,
NULL);

for (int i = 0; i < ACCEPT_POSTED; i++)
{
    accept_ctx* a = (accept_ctx*)calloc(1, sizeof(accept_ctx));
    if (a)
        post_accept(a);
}

SetConsoleCtrlHandler(ctrl_handler, TRUE);
log_line("Сервер запущен, порт %u, рабочих потоков: %d (Ctrl+C -
выход)",
    port, nthreads);

while (!g_stop)
{
    Sleep(1000);
}

```

```
    watchdog();  
}  
  
log_line("Остановка сервера...");  
closesocket(g_listen); /* незавершённые AcceptEx вернутся с  
ошибкой */  
for (int i = 0; i < nthreads; i++)  
    PostQueuedCompletionStatus(g_port, 0, KEY_STOP, NULL);  
WaitForMultipleObjects((DWORD)nthreads, threads, TRUE, 5000);  
  
crypto_cleanup();  
WSACleanup();  
return 0;  
}
```

Приложение Б

sysinfo.c

```
#include "common.h"
#include <aclapi.h>
#include <sddl.h>

/*
-----
*/
/* Вспомогательное
*/
/*
-----
*/

static void result_ok(sbuf* r, const char* cmd)
{
    sb_rec_begin(r, "RESULT");
    sb_kv_str(r, "status", "OK");
    sb_kv_str(r, "command", cmd);
    sb_rec_end(r);
}

static char* win_error_text(DWORD code)
{
    wchar_t buf[512];
    DWORD n = FormatMessageW(FORMAT_MESSAGE_FROM_SYSTEM |
        FORMAT_MESSAGE_IGNORE_INSERTS,
        NULL, code, 0, buf, 512, NULL);
    while (n > 0 &&
        (buf[n - 1] == L'\r' || buf[n - 1] == L'\n' || buf[n - 1]
        == L' '))
        n--;
    buf[n] = 0;
    if (n == 0)
        return _strdup("unknown error");
    return wide_to_utf8(buf);
}
```

```

/* ошибка протокола (source=protocol) */
static int result_proto_error(sbuf* r, int code, const char* msg)
{
    r->len = 0;
    r->data[0] = 0;
    sb_rec_begin(r, "RESULT");
    sb_kv_str(r, "status", "ERROR");
    sb_kv_i64(r, "code", code);
    sb_kv_str(r, "source", "protocol");
    sb_kv_str(r, "message", msg);
    sb_rec_end(r);
    return 0;
}

```

```

/* ошибка Win32 (source=win32) */
static int result_win_error(sbuf* r, DWORD code)
{
    char* m = win_error_text(code);
    r->len = 0;
    r->data[0] = 0;
    sb_rec_begin(r, "RESULT");
    sb_kv_str(r, "status", "ERROR");
    sb_kv_i64(r, "code", (__int64)code);
    sb_kv_str(r, "source", "win32");
    sb_kv_str(r, "message", m);
    sb_rec_end(r);
    free(m);
    return 0;
}

```

```

static unsigned __int64 filetime_to_unix(const FILETIME* ft)
{
    ULARGE_INTEGER u;
    u.LowPart = ft->dwLowDateTime;
    u.HighPart = ft->dwHighDateTime;
    return (u.QuadPart - 116444736000000000ULL) / 10000000ULL;
}

```

```

static int reg_read_sz(const wchar_t* sub, const wchar_t* name,
wchar_t* out,

```

```

        DWORD cch)
{
    HKEY k;
    DWORD type = 0, sz = cch * sizeof(wchar_t);
    int ok = 0;
    if (RegOpenKeyExW(HKEY_LOCAL_MACHINE, sub, 0, KEY_READ |
KEY_WOW64_64KEY,
        &k) == ERROR_SUCCESS)
    {
        if (RegQueryValueExW(k, name, NULL, &type, (BYTE*)out, &sz) ==
            ERROR_SUCCESS &&
            (type == REG_SZ))
        {
            out[cch - 1] = 0;
            ok = 1;
        }
        RegCloseKey(k);
    }
    if (!ok)
        out[0] = 0;
    return ok;
}

/*
-----
*/
/* GET_OS
*/
/*
-----
*/
typedef LONG(WINAPI* RtlGetVersion_fn)(RTL_OSVERSIONINFOEXW*);

static const char* friendly_os_name(DWORD maj, DWORD min, DWORD
build,
        int workstation)
{
    if (maj == 10)
    {
        if (workstation)

```

```

        return build >= 22000 ? "Windows 11" : "Windows 10";
    if (build >= 26100)
        return "Windows Server 2025";
    if (build >= 20348)
        return "Windows Server 2022";
    if (build >= 17763)
        return "Windows Server 2019";
    return "Windows Server 2016";
}
if (maj == 6 && min == 3)
    return workstation ? "Windows 8.1" : "Windows Server 2012 R2";
if (maj == 6 && min == 2)
    return workstation ? "Windows 8" : "Windows Server 2012";
if (maj == 6 && min == 1)
    return workstation ? "Windows 7" : "Windows Server 2008 R2";
if (maj == 6 && min == 0)
    return workstation ? "Windows Vista" : "Windows Server 2008";
if (maj == 5 && min == 2)
    return "Windows Server 2003 / XP x64";
if (maj == 5 && min == 1)
    return "Windows XP";
if (maj == 5 && min == 0)
    return "Windows 2000";
return "Windows (unknown)";
}

static int cmd_os(sbuf* r)
{
    /* GetVersionEx на Windows 8.1+ без манифеста врёт, поэтому
    берём
    * RtlGetVersion из ntdll */
    RTL_OSVERSIONINFOEXW v;
    RtlGetVersion_fn f = (RtlGetVersion_fn)GetProcAddress(
        GetModuleHandleW(L"ntdll.dll"), "RtlGetVersion");
    memset(&v, 0, sizeof(v));
    v.dwOSVersionInfoSize = sizeof(v);
    if (!f || f(&v) != 0)
        return result_proto_error(r, 100, "cannot determine OS
version");
}

```

```

SYSTEM_INFO si;
GetNativeSystemInfo(&si);
const char* arch = "unknown";
switch (si.wProcessorArchitecture)
{
case PROCESSOR_ARCHITECTURE_AMD64:
    arch = "x64";
    break;
case PROCESSOR_ARCHITECTURE_INTEL:
    arch = "x86";
    break;
case PROCESSOR_ARCHITECTURE_ARM:
    arch = "arm";
    break;
case 12:
    arch = "arm64";
    break;
}

int workstation = (v.wProductType == VER_NT_WORKSTATION);
const char* ptype =
    workstation
        ? "workstation"
        : (v.wProductType == VER_NT_DOMAIN_CONTROLLER ?
"domain_controller"
        : "server");

wchar_t product[256], dispver[64], comp[MAX_COMPUTERNAME_LENGTH
+ 1];
DWORD csz = MAX_COMPUTERNAME_LENGTH + 1;
const wchar_t* key = L"SOFTWARE\\Microsoft\\Windows NT\\
\\CurrentVersion";
reg_read_sz(key, L"ProductName", product, 256);
if (!reg_read_sz(key, L"DisplayVersion", dispver, 64))
    reg_read_sz(key, L"ReleaseId", dispver, 64);
if (!GetComputerNameW(comp, &csz))
    comp[0] = 0;

result_ok(r, "GET_OS");
sb_rec_begin(r, "OS");

```

```

    sb_kv_str(r, "name",
        friendly_os_name(v.dwMajorVersion, v.dwMinorVersion,
            v.dwBuildNumber, workstation));
    sb_kv_wstr(r, "product_name", product);
    sb_kv_wstr(r, "display_version", dispver);
    sb_kv_u64(r, "major", v.dwMajorVersion);
    sb_kv_u64(r, "minor", v.dwMinorVersion);
    sb_kv_u64(r, "build", v.dwBuildNumber);
    sb_kv_u64(r, "platform_id", v.dwPlatformId);
    sb_kv_wstr(r, "service_pack", v.szCSDVersion);
    sb_kv_u64(r, "sp_major", v.wServicePackMajor);
    sb_kv_u64(r, "sp_minor", v.wServicePackMinor);
    sb_kv_str(r, "product_type", ptype);
    sb_kv_str(r, "arch", arch);
    sb_kv_wstr(r, "computer_name", comp);
    sb_rec_end(r);
    return 1;
}

/*
-----
*/
/* GET_TIME / GET_UPTIME
*/
/*
-----
*/
static void time_record(sbuf* r, const char* scope, const
SYSTEMTIME* t,
    unsigned __int64 unix_s, const char* offset_iso,
    int has_tz, LONG offset_min, const wchar_t* tzname)
{
    char iso[64];
    sprintf(iso, "%04u-%02u-%02uT%02u:%02u:%02u.%03u%s", t->wYear,
t->wMonth,
        t->wDay, t->wHour, t->wMinute, t->wSecond, t->wMilliseconds,
        offset_iso);
    sb_rec_begin(r, "TIME");
    sb_kv_str(r, "scope", scope);
    sb_kv_str(r, "iso", iso);

```

```

    sb_kv_u64(r, "year", t->wYear);
    sb_kv_u64(r, "month", t->wMonth);
    sb_kv_u64(r, "day", t->wDay);
    sb_kv_u64(r, "hour", t->wHour);
    sb_kv_u64(r, "minute", t->wMinute);
    sb_kv_u64(r, "second", t->wSecond);
    sb_kv_u64(r, "millisecond", t->wMilliseconds);
    sb_kv_u64(r, "day_of_week", t->wDayOfWeek); /* 0 = воскресенье
*/
    sb_kv_u64(r, "unix", unix_s);
    if (has_tz)
    {
        sb_kv_i64(r, "utc_offset_minutes", offset_min);
        sb_kv_wstr(r, "tz_name", tzname);
    }
    sb_rec_end(r);
}

static int cmd_time(sbuf* r)
{
    SYSTEMTIME ut, lt;
    FILETIME ft;
    GetSystemTime(&ut);
    GetLocalTime(&lt);
    SystemTimeToFileTime(&ut, &ft);
    unsigned __int64 unix_s = filetime_to_unix(&ft);

    TIME_ZONE_INFORMATION tz;
    DWORD tztz = GetTimeZoneInformation(&tz);
    LONG bias = tz.Bias;
    const wchar_t* tzname = tz.StandardName;
    if (tztz == TIME_ZONE_ID_DAYLIGHT)
    {
        bias += tz.DaylightBias;
        tzname = tz.DaylightName;
    }
    else if (tztz != TIME_ZONE_ID_INVALID)
    {
        bias += tz.StandardBias;
    }
}

```

```

    LONG offset = -bias;
    char off_iso[16];
    sprintf(off_iso, "%c%02ld:%02ld", offset < 0 ? '-' : '+',
labs(offset) / 60,
    labs(offset) % 60);

    result_ok(r, "GET_TIME");
    time_record(r, "utc", &ut, unix_s, "Z", 0, 0, NULL);
    time_record(r, "local", &lt, unix_s, off_iso, 1, offset,
tzname);
    return 1;
}

static int cmd_uptime(sbuf* r)
{
    unsigned __int64 ms = GetTickCount64(); /* в отличие от
GetTickCount не
                                переполняется за 49 суток */

    SYSTEMTIME now;
    FILETIME ft;
    GetSystemTime(&now);
    SystemTimeToFileTime(&now, &ft);

    ULARGE_INTEGER u;
    u.LowPart = ft.dwLowDateTime;
    u.HighPart = ft.dwHighDateTime;
    u.QuadPart -= ms * 10000ULL;
    ft.dwLowDateTime = u.LowPart;
    ft.dwHighDateTime = u.HighPart;
    SYSTEMTIME boot;
    FileTimeToSystemTime(&ft, &boot);
    char iso[64];
    sprintf(iso, "%04u-%02u-%02uT%02u:%02u:%02uZ", boot.wYear,
boot.wMonth,
    boot.wDay, boot.wHour, boot.wMinute, boot.wSecond);

    result_ok(r, "GET_UPTIME");
    sb_rec_begin(r, "UPTIME");
    sb_kv_u64(r, "milliseconds", ms);
    sb_kv_str(r, "boot_time_utc", iso);

```

```

    sb_kv_u64(r, "boot_time_unix", filetime_to_unix(&ft));
    sb_rec_end(r);
    return 1;
}

/*
-----
*/
/* GET_MEMORY
*/
/*
-----
*/
static int cmd_memory(sbuf* r)
{
    MEMORYSTATUSEX m;
    m.dwLength = sizeof(m);
    if (!GlobalMemoryStatusEx(&m))
        return result_win_error(r, GetLastError());

    result_ok(r, "GET_MEMORY");
    sb_rec_begin(r, "MEMORY");
    sb_kv_u64(r, "load_percent", m.dwMemoryLoad);
    sb_kv_u64(r, "total_phys", m.ullTotalPhys);
    sb_kv_u64(r, "avail_phys", m.ullAvailPhys);
    sb_kv_u64(r, "total_pagefile", m.ullTotalPageFile);
    sb_kv_u64(r, "avail_pagefile", m.ullAvailPageFile);
    sb_kv_u64(r, "total_virtual", m.ullTotalVirtual);
    sb_kv_u64(r, "avail_virtual", m.ullAvailVirtual);
    sb_rec_end(r);
    return 1;
}

/*
-----
*/
/* GET_DRIVES / GET_FREESPACE
*/
/*
-----
*/

```

```

*/
static void drive_type_names(UINT t, const char** type, const
char** cls)
{
    switch (t)
    {
        case DRIVE_REMOVABLE:
            *type = "removable";
            *cls = "removable";
            break;
        case DRIVE_FIXED:
            *type = "fixed";
            *cls = "local";
            break;
        case DRIVE_REMOTE:
            *type = "network";
            *cls = "network";
            break;
        case DRIVE_CDROM:
            *type = "cdrom";
            *cls = "removable";
            break;
        case DRIVE_RAMDISK:
            *type = "ramdisk";
            *cls = "local";
            break;
        case DRIVE_NO_ROOT_DIR:
            *type = "no_root";
            *cls = "unknown";
            break;
        default:
            *type = "unknown";
            *cls = "unknown";
            break;
    }
}

static int cmd_drives(sbuf* r)
{
    wchar_t buf[512];

```

```

DWORD n = GetLogicalDriveStringsW(512, buf);
if (n == 0 || n > 512)
    return result_win_error(r, GetLastError());

result_ok(r, "GET_DRIVES");
for (const wchar_t* p = buf; *p; p += wcslen(p) + 1)
{
    const char *type, *cls;
    wchar_t label[MAX_PATH + 1] = L"", fs[MAX_PATH + 1] = L"";
    DWORD serial = 0, maxc = 0, flags = 0;
    UINT dt = GetDriveTypeW(p);
    drive_type_names(dt, &type, &cls);
    int ready = GetVolumeInformationW(p, label, MAX_PATH + 1,
&serial,
                                &maxc, &flags, fs, MAX_PATH + 1);
    sb_rec_begin(r, "DRIVE");
    sb_kv_wstr(r, "root", p);
    sb_kv_str(r, "type", type);
    sb_kv_str(r, "class", cls);
    sb_kv_wstr(r, "fs", ready ? fs : L "");
    sb_kv_wstr(r, "label", ready ? label : L "");
    sb_kv_str(r, "ready", ready ? "1" : "0");
    sb_rec_end(r);
}
return 1;
}

static int cmd_freespace(sbuf* r)
{
    wchar_t buf[512];
    DWORD n = GetLogicalDriveStringsW(512, buf);
    if (n == 0 || n > 512)
        return result_win_error(r, GetLastError());

    result_ok(r, "GET_FREESPACE");
    for (const wchar_t* p = buf; *p; p += wcslen(p) + 1)
    {
        if (GetDriveTypeW(p) != DRIVE_FIXED)
            continue; /* только локальные диски */
        ULARGE_INTEGER avail, total, freeb;

```

```

    sb_rec_begin(r, "FREESPACE");
    sb_kv_wstr(r, "root", p);
    if (GetDiskFreeSpaceExW(p, &avail, &total, &freeb))
    {
        sb_kv_u64(r, "total_bytes", total.QuadPart);
        sb_kv_u64(r, "free_bytes", freeb.QuadPart);
        sb_kv_u64(r, "free_to_user_bytes", avail.QuadPart);
    }
    else
    {
        sb_kv_i64(r, "error", (__int64)GetLastError());
    }
    sb_rec_end(r);
}
return 1;
}

/*
-----
*/
/* Права доступа и владелец
*/
/*
-----
*/
typedef enum
{
    OBJ_FILE,
    OBJ_DIR,
    OBJ_REG
} objkind;

/* Разбор типа и пути; возвращает имя для GetNamedSecurityInfo
(malloc) */
static wchar_t* resolve_object(const char* kind, const char* path,
                               SE_OBJECT_TYPE* se, objkind* ok_kind,
                               const char** err)
{
    wchar_t* w = utf8_to_wide(path);
    if (_stricmp(kind, "FILE") == 0)

```

```

{
    *se = SE_FILE_OBJECT;
    DWORD a = GetFileAttributesW(w);
    *ok_kind =
        (a != INVALID_FILE_ATTRIBUTES && (a &
FILE_ATTRIBUTE_DIRECTORY))
        ? OBJ_DIR
        : OBJ_FILE;
    return w;
}
if (_stricmp(kind, "REG") == 0)
{
    static const struct
    {
        const wchar_t* in;
        const wchar_t* out;
    } roots[] = {
        {L"HKLM", L"MACHINE"},
        {L"HKEY_LOCAL_MACHINE", L"MACHINE"},
        {L"MACHINE", L"MACHINE"},
        {L"HKCU", L"CURRENT_USER"},
        {L"HKEY_CURRENT_USER", L"CURRENT_USER"},
        {L"CURRENT_USER", L"CURRENT_USER"},
        {L"HKCR", L"CLASSES_ROOT"},
        {L"HKEY_CLASSES_ROOT", L"CLASSES_ROOT"},
        {L"CLASSES_ROOT", L"CLASSES_ROOT"},
        {L"HKU", L"USERS"},
        {L"HKEY_USERS", L"USERS"},
        {L"USERS", L"USERS"},
    };
    wchar_t* rest = wcschr(w, L'\\');
    size_t rootlen = rest ? (size_t)(rest - w) : wcslen(w);
    for (size_t i = 0; i < sizeof(roots) / sizeof(roots[0]); i++)
    {
        if (wcslen(roots[i].in) == rootlen &&
            _wcsnicmp(w, roots[i].in, rootlen) == 0)
        {
            size_t cap =
                wcslen(roots[i].out) + (rest ? wcslen(rest) : 0) + 1;
            wchar_t* res = (wchar_t*)malloc(cap * sizeof(wchar_t));

```

```

        wcsncpy(res, roots[i].out);
        if (rest)
            wscat(res, rest);
        free(w);
        *se = SE_REGISTRY_KEY;
        *ok_kind = OBJ_REG;
        return res;
    }
}
free(w);
*err = "unknown registry root (use HKLM, HKCU, HKCR, HKU)";
return NULL;
}
free(w);
*err = "object type must be FILE or REG";
return NULL;
}

```

/* SID -> строка и имя */

```

static void sid_describe(PSID sid, char** sidstr, char** name,
const char** use)
{
    LPWSTR s = NULL;
    *sidstr = ConvertSidToStringSidW(sid, &s) ? wide_to_utf8(s) :
_strdup("");
    if (s)
        LocalFree(s);

    wchar_t nm[256], dom[256];
    DWORD nl = 256, dl = 256;
    SID_NAME_USE u;
    if (LookupAccountSidW(NULL, sid, nm, &nl, dom, &dl, &u))
    {
        wchar_t full[600];
        if (dom[0])
            swprintf(full, 600, L"%ls\\%ls", dom, nm);
        else
            swprintf(full, 600, L"%ls", nm);
        *name = wide_to_utf8(full);
        switch (u)

```

```

{
case SidTypeUser:
    *use = "user";
    break;
case SidTypeGroup:
    *use = "group";
    break;
case SidTypeDomain:
    *use = "domain";
    break;
case SidTypeAlias:
    *use = "alias";
    break;
case SidTypeWellKnownGroup:
    *use = "well_known_group";
    break;
case SidTypeDeletedAccount:
    *use = "deleted_account";
    break;
case SidTypeInvalid:
    *use = "invalid";
    break;
case SidTypeComputer:
    *use = "computer";
    break;
case SidTypeLabel:
    *use = "label";
    break;
default:
    *use = "unknown";
    break;
}
}
else
{
    *name = _strdup("");
    *use = "unresolved";
}
}

```

```

static int cmd_owner(sbuf* r, const char* kind, const char* path)
{
    SE_OBJECT_TYPE se;
    objkind ok;
    const char* err = NULL;
    wchar_t* obj = resolve_object(kind, path, &se, &ok, &err);
    if (!obj)
        return result_proto_error(r, 2, err);

    PSID owner = NULL;
    PSECURITY_DESCRIPTOR sd = NULL;
    DWORD e = GetNamedSecurityInfoW(obj, se,
OWNER_SECURITY_INFORMATION, &owner,
        NULL, NULL, NULL, &sd);

    free(obj);
    if (e != ERROR_SUCCESS)
        return result_win_error(r, e);
    if (!owner)
    {
        LocalFree(sd);
        return result_proto_error(r, 101, "object has no owner");
    }

    char *sidstr, *name;
    const char* use;
    sid_describe(owner, &sidstr, &name, &use);
    result_ok(r, "GET_OWNER");
    sb_rec_begin(r, "OWNER");
    sb_kv_str(r, "object_type",
        ok == OBJ_REG ? "registry"
        : (ok == OBJ_DIR ? "directory" : "file"));
    sb_kv_str(r, "path", path);
    sb_kv_str(r, "sid", sidstr);
    sb_kv_str(r, "name", name);
    sb_kv_str(r, "name_use", use);
    sb_rec_end(r);
    free(sidstr);
    free(name);
    LocalFree(sd);
    return 1;
}

```

```
}
```

```
/* названия битов маски доступа (MSDN: File / Registry Key  
Security and Access
```

```
* Rights) */
```

```
static const struct
```

```
{
```

```
    int bit;
```

```
    const char *file, *dir, *reg;
```

```
} g_bits[] = {
```

```
    {0, "FILE_READ_DATA", "FILE_LIST_DIRECTORY", "KEY_QUERY_VALUE"},
```

```
    {1, "FILE_WRITE_DATA", "FILE_ADD_FILE", "KEY_SET_VALUE"},
```

```
    {2, "FILE_APPEND_DATA", "FILE_ADD_SUBDIRECTORY",
```

```
"KEY_CREATE_SUB_KEY"},
```

```
    {3, "FILE_READ_EA", "FILE_READ_EA", "KEY_ENUMERATE_SUB_KEYS"},
```

```
    {4, "FILE_WRITE_EA", "FILE_WRITE_EA", "KEY_NOTIFY"},
```

```
    {5, "FILE_EXECUTE", "FILE_TRAVERSE", "KEY_CREATE_LINK"},
```

```
    {6, "FILE_DELETE_CHILD", "FILE_DELETE_CHILD", NULL},
```

```
    {7, "FILE_READ_ATTRIBUTES", "FILE_READ_ATTRIBUTES", NULL},
```

```
    {8, "FILE_WRITE_ATTRIBUTES", "FILE_WRITE_ATTRIBUTES", NULL},
```

```
    {16, "DELETE", "DELETE", "DELETE"},
```

```
    {17, "READ_CONTROL", "READ_CONTROL", "READ_CONTROL"},
```

```
    {18, "WRITE_DAC", "WRITE_DAC", "WRITE_DAC"},
```

```
    {19, "WRITE_OWNER", "WRITE_OWNER", "WRITE_OWNER"},
```

```
    {20, "SYNCHRONIZE", "SYNCHRONIZE", "SYNCHRONIZE"},
```

```
    {24, "ACCESS_SYSTEM_SECURITY", "ACCESS_SYSTEM_SECURITY",
```

```
        "ACCESS_SYSTEM_SECURITY"},
```

```
    {25, "MAXIMUM_ALLOWED", "MAXIMUM_ALLOWED", "MAXIMUM_ALLOWED"},
```

```
    {28, "GENERIC_ALL", "GENERIC_ALL", "GENERIC_ALL"},
```

```
    {29, "GENERIC_EXECUTE", "GENERIC_EXECUTE", "GENERIC_EXECUTE"},
```

```
    {30, "GENERIC_WRITE", "GENERIC_WRITE", "GENERIC_WRITE"},
```

```
    {31, "GENERIC_READ", "GENERIC_READ", "GENERIC_READ"},
```

```
};
```

```
static void mask_bits(DWORD mask, objkind k, sbuf* bits, sbuf*  
names)
```

```
{
```

```
    int first = 1;
```

```
    for (int b = 0; b < 32; b++)
```

```
    {
```

```

    if (!(mask & (1UL << b)))
        continue;
    const char* nm = NULL;
    for (size_t i = 0; i < sizeof(g_bits) / sizeof(g_bits[0]); i++)
    {
        if (g_bits[i].bit == b)
        {
            nm = (k == OBJ_FILE)
                ? g_bits[i].file
                : (k == OBJ_DIR ? g_bits[i].dir : g_bits[i].reg);
            break;
        }
    }
    char tmp[48];
    if (!nm)
    {
        sprintf(tmp, "UNKNOWN_BIT_%d", b);
        nm = tmp;
    }
    if (!first)
    {
        sb_append(bits, ",");
        sb_append(names, ",");
    }
    sb_printf(bits, "%d", b);
    sb_append(names, nm);
    first = 0;
}
}

```

```

static const char* g_ace_types[] = {
    "ACCESS_ALLOWED_ACE_TYPE",
    "ACCESS_DENIED_ACE_TYPE",
    "SYSTEM_AUDIT_ACE_TYPE",
    "SYSTEM_ALARM_ACE_TYPE",
    "ACCESS_ALLOWED_COMPOUND_ACE_TYPE",
    "ACCESS_ALLOWED_OBJECT_ACE_TYPE",
    "ACCESS_DENIED_OBJECT_ACE_TYPE",
    "SYSTEM_AUDIT_OBJECT_ACE_TYPE",
}

```

```

"SYSTEM_ALARM_OBJECT_ACE_TYPE",
"ACCESS_ALLOWED_CALLBACK_ACE_TYPE",
"ACCESS_DENIED_CALLBACK_ACE_TYPE",
"ACCESS_ALLOWED_CALLBACK_OBJECT_ACE_TYPE",
"ACCESS_DENIED_CALLBACK_OBJECT_ACE_TYPE",
"SYSTEM_AUDIT_CALLBACK_ACE_TYPE",
"SYSTEM_ALARM_CALLBACK_ACE_TYPE",
"SYSTEM_AUDIT_CALLBACK_OBJECT_ACE_TYPE",
"SYSTEM_ALARM_CALLBACK_OBJECT_ACE_TYPE",
"SYSTEM_MANDATORY_LABEL_ACE_TYPE",
"SYSTEM_RESOURCE_ATTRIBUTE_ACE_TYPE",
"SYSTEM_SCOPED_POLICY_ID_ACE_TYPE",
};

/* Где в ACE лежит SID. Возвращает NULL, если формат не
поддерживается. */
static PSID ace_sid(const BYTE* ace, DWORD* mask)
{
    const ACE_HEADER* h = (const ACE_HEADER*)ace;
    if (h->AceSize < 8)
        return NULL;
    *mask = *(const DWORD*)(ace + 4);
    switch (h->AceType)
    {
    case 0:
    case 1:
    case 2:
    case 3: /* ALLOWED / DENIED / AUDIT / ALARM */
    case 9:
    case 10:
    case 13:
    case 14: /* CALLBACK-варианты */
    case 17:
    case 18:
    case 19:
        return (PSID)(ace + 8);
    case 5:
    case 6:
    case 7:
    case 8: /* OBJECT-варианты */

```

```

case 11:
case 12:
case 15:
case 16:
{
    if (h->AceSize < 12)
        return NULL;
    DWORD flags = *(const DWORD*)(ace + 8);
    DWORD off = 12;
    if (flags & ACE_OBJECT_TYPE_PRESENT)
        off += 16;
    if (flags & ACE_INHERITED_OBJECT_TYPE_PRESENT)
        off += 16;
    if (off >= h->AceSize)
        return NULL;
    return (PSID)(ace + off);
}
default:
    return NULL;
}
}

/* Область действия ACE (как в диалоге "Применяется к") */
static const char* ace_scope(BYTE fl)
{
    int oi = (fl & OBJECT_INHERIT_ACE) != 0;
    int ci = (fl & CONTAINER_INHERIT_ACE) != 0;
    int io = (fl & INHERIT_ONLY_ACE) != 0;
    if (oi && ci)
        return io ? "subfolders_and_files_only"
            : "this_folder_subfolders_and_files";
    if (ci)
        return io ? "subfolders_only" : "this_folder_and_subfolders";
    if (oi)
        return io ? "files_only" : "this_folder_and_files";
    return "this_object_only";
}

static void ace_flag_names(BYTE fl, sbuf* s)
{

```

```

static const struct
{
    BYTE bit;
    const char* n;
} t[] = {
    {OBJECT_INHERIT_ACE, "OBJECT_INHERIT_ACE"},
    {CONTAINER_INHERIT_ACE, "CONTAINER_INHERIT_ACE"},
    {NO_PROPAGATE_INHERIT_ACE, "NO_PROPAGATE_INHERIT_ACE"},
    {INHERIT_ONLY_ACE, "INHERIT_ONLY_ACE"},
    {INHERITED_ACE, "INHERITED_ACE"},
};
int first = 1;
for (size_t i = 0; i < sizeof(t) / sizeof(t[0]); i++)
{
    if (fl & t[i].bit)
    {
        if (!first)
            sb_append(s, ",");
        sb_append(s, t[i].n);
        first = 0;
    }
}
}

static int cmd_acl(sbuf* r, const char* kind, const char* path)
{
    SE_OBJECT_TYPE se;
    objkind ok;
    const char* err = NULL;
    wchar_t* obj = resolve_object(kind, path, &se, &ok, &err);
    if (!obj)
        return result_proto_error(r, 2, err);

    PACL dacl = NULL;
    PSECURITY_DESCRIPTOR sd = NULL;
    DWORD e = GetNamedSecurityInfoW(obj, se,
    DACL_SECURITY_INFORMATION, NULL,
        NULL, &dacl, NULL, &sd);
    free(obj);
    if (e != ERROR_SUCCESS)

```

```

    return result_win_error(r, e);

SECURITY_DESCRIPTOR_CONTROL ctl = 0;
DWORD rev;
GetSecurityDescriptorControl(sd, &ctl, &rev);

DWORD count = 0;
if (dacl)
{
    ACL_SIZE_INFORMATION info;
    if (GetAclInformation(dacl, &info, sizeof(info),
AclSizeInformation))
        count = info.AceCount;
}

result_ok(r, "GET_ACL");
sb_rec_begin(r, "ACL");
sb_kv_str(r, "object_type",
    ok == OBJ_REG ? "registry"
        : (ok == OBJ_DIR ? "directory" : "file"));
sb_kv_str(r, "path", path);
sb_kv_str(r, "null_dacl",
    dacl ? "0" : "1"); /* NULL DACL = полный доступ всем */
sb_kv_str(r, "dacl_protected", (ctl & SE_DACL_PROTECTED) ? "1" :
"0");
sb_kv_u64(r, "ace_count", count);
sb_rec_end(r);

for (DWORD i = 0; i < count; i++)
{
    void* pace = NULL;
    if (!GetAce(dacl, i, &pace) || !pace)
        continue;
    const ACE_HEADER* h = (const ACE_HEADER*)pace;
    DWORD mask = 0;
    PSID sid = ace_sid((const BYTE*)pace, &mask);

    sb_rec_begin(r, "ACE");
    sb_kv_u64(r, "index", i);
    if (sid && IsValidSid(sid))

```

```

{
    char *sidstr, *name;
    const char* use;
    sid_describe(sid, &sidstr, &name, &use);
    sb_kv_str(r, "sid", sidstr);
    sb_kv_str(r, "name", name);
    sb_kv_str(r, "name_use", use);
    free(sidstr);
    free(name);
}
else
{
    sb_kv_str(r, "sid", "");
    sb_kv_str(r, "name", "");
    sb_kv_str(r, "name_use", "unsupported");
}
sb_kv_str(r, "ace_type",
          h->AceType < sizeof(g_ace_types) /
sizeof(g_ace_types[0])
          ? g_ace_types[h->AceType]
          : "UNKNOWN_ACE_TYPE");
sb_kv_u64(r, "ace_type_id", h->AceType);
sb_kv_hex(r, "ace_flags", h->AceFlags);
sbuf fl;
sb_init(&fl);
ace_flag_names(h->AceFlags, &fl);
sb_kv_str(r, "ace_flag_names", fl.data);
sb_free(&fl);
sb_kv_str(r, "scope", ace_scope(h->AceFlags));
sb_kv_hex(r, "mask", mask);
sbuf bits, names;
sb_init(&bits);
sb_init(&names);
mask_bits(mask, ok, &bits, &names);
sb_kv_str(r, "bits", bits.data);
sb_kv_str(r, "bit_names", names.data);
sb_free(&bits);
sb_free(&names);
sb_rec_end(r);
}

```

```

    LocalFree(sd);
    return 1;
}

/*
-----
*/
/* Разбор запроса
*/
/*
-----
*/
#define MAX_ARGS 3

/* Разбить запрос на токены и снять экранирование. Возвращает
копию (free),
 * токены указывают в неё. */
static char* split_request(const char* req, char* tok[MAX_ARGS],
int* ntok)
{
    char* copy = _strdup(req);
    *ntok = 0;
    if (!copy)
        return NULL;
    size_t l = strlen(copy);
    while (l > 0 && (copy[l - 1] == '\n' || copy[l - 1] == '\r'))
        copy[--l] = 0;
    char* p = copy;
    while (p && *ntok < MAX_ARGS)
    {
        char* next = strchr(p, '\t');
        if (next)
            *next++ = 0;
        unescape_inplace(p);
        tok[(*ntok)++] = p;
        p = next;
    }
    return copy;
}

```

```

void request_summary(const char* req, char* out, size_t outsz)
{
    char* tok[MAX_ARGS];
    int n;
    char* copy = split_request(req, tok, &n);
    out[0] = 0;
    if (!copy)
        return;
    for (int i = 0; i < n; i++)
    {
        if (i)
            strncat(out, " ", outsz - strlen(out) - 1);
        strncat(out, tok[i], outsz - strlen(out) - 1);
    }
    free(copy);
}

```

```

int process_request(const char* req, sbuf* r)
{
    char* tok[MAX_ARGS];
    int n;
    char* copy = split_request(req, tok, &n);
    int res;
    if (!copy || n == 0 || tok[0][0] == 0)
    {
        free(copy);
        return result_proto_error(r, 1, "empty request");
    }

```

```

    const char* cmd = tok[0];
    if (_stricmp(cmd, "GET_OS") == 0)
        res = cmd_os(r);
    else if (_stricmp(cmd, "GET_TIME") == 0)
        res = cmd_time(r);
    else if (_stricmp(cmd, "GET_UPTIME") == 0)
        res = cmd_uptime(r);
    else if (_stricmp(cmd, "GET_MEMORY") == 0)
        res = cmd_memory(r);
    else if (_stricmp(cmd, "GET_DRIVES") == 0)
        res = cmd_drives(r);

```

```

else if (_stricmp(cmd, "GET_FREESPACE") == 0)
    res = cmd_freespace(r);
else if (_stricmp(cmd, "GET_ACL") == 0 || _stricmp(cmd,
"GET_OWNER") == 0)
{
    if (n < 3 || tok[2][0] == 0)
    {
        res =
            result_proto_error(r, 2, "usage: <command> <FILE|REG>
<path>");
    }
    else if (_stricmp(cmd, "GET_ACL") == 0)
    {
        res = cmd_acl(r, tok[1], tok[2]);
    }
    else
    {
        res = cmd_owner(r, tok[1], tok[2]);
    }
}
else
{
    res = result_proto_error(r, 1, "unknown command");
}
free(copy);
return res;
}

```

Приложение В

client.c

```
#include "common.h"
#include <ws2tcpip.h>

static HCRYPTKEY g_pair;
static BYTE* g_pub;
static DWORD g_publen;
static char g_host[256] = "127.0.0.1";
static unsigned short g_port = PROTO_PORT;
static int g_raw;

/*
-----
*/
/* Сеть
*/
/*
-----
*/
static SOCKET connect_to(const char* host, unsigned short port)
{
    struct addrinfo hints, *res = NULL, *p;
    char portstr[16];
    SOCKET s = INVALID_SOCKET;

    memset(&hints, 0, sizeof(hints));
    hints.ai_family = AF_INET;
    hints.ai_socktype = SOCK_STREAM;
    sprintf(portstr, "%u", port);
    if (getaddrinfo(host, portstr, &hints, &res) != 0)
    {
        con_print("Не удалось определить адрес \"%s\"\n", host);
        return INVALID_SOCKET;
    }
    for (p = res; p; p = p->ai_next)
    {
        s = socket(p->ai_family, p->ai_socktype, p->ai_protocol);
```

```

    if (s == INVALID_SOCKET)
        continue;
    if (connect(s, p->ai_addr, (int)p->ai_addrlen) == 0)
        break;
    closesocket(s);
    s = INVALID_SOCKET;
}
freeaddrinfo(res);
if (s == INVALID_SOCKET)
{
    con_print("Не удалось подключиться к %s:%u (ошибка %d)\n",
host, port,
        WSAGetLastError());
    return INVALID_SOCKET;
}
DWORD tmo = 30000;
setsockopt(s, SOL_SOCKET, SO_RCVTIMEO, (const char*)&tmo,
sizeof(tmo));
return s;
}

/* Выполнить запрос. При успехе возвращает расшифрованный текст
ответа (free),
* иначе NULL. */
static char* do_request(const char* req)
{
    SOCKET s = connect_to(g_host, g_port);
    if (s == INVALID_SOCKET)
        return NULL;

    BYTE *keyblob = NULL, *enc = NULL, *resp = NULL, *plain = NULL;
    HCRYPTKEY sess = 0;
    DWORD kl = 0, el = 0, rl = 0, pl = 0;
    char* result = NULL;

    if (!net_send_frame(s, g_pub, g_publen))
    {
        con_print("Ошибка передачи публичного ключа\n");
        goto done;
    }
}

```

```

keyblob = net_recv_frame(s, 4096, &kl);
if (!keyblob)
{
    con_print("Не получен сеансовый ключ от сервера\n");
    goto done;
}
if (!crypto_import_session_key(g_pair, keyblob, kl, &sess))
    goto done;

enc = crypto_encrypt(sess, (const BYTE*)req, (DWORD)strlen(req),
&el);
if (!enc || !net_send_frame(s, enc, el))
{
    con_print("Ошибка отправки запроса\n");
    goto done;
}
resp = net_recv_frame(s, CLIENT_RX_MAX, &rl);
if (!resp)
{
    con_print(
        "Ответ не получен (сервер закрыл соединение или истёк таймаут)\n");
    goto done;
}
plain = crypto_decrypt(sess, resp, rl, &pl);
if (!plain)
{
    con_print("Не удалось расшифровать ответ\n");
    goto done;
}
result = (char*)plain;
plain = NULL;
done:
free(keyblob);
free(enc);
free(resp);
free(plain);
crypto_free_key(sess);
closesocket(s);
return result;

```

```

}

/*
-----
*/
/* Форматирование
*/
/*
-----
*/
static const char* get(const record* r, const char* k)
{
    const char* v = rec_get(r, k);
    return v ? v : "";
}

static unsigned __int64 getu(const record* r, const char* k)
{
    return _strtoui64(get(r, k), NULL, 0);
}

static void fmt_bytes(unsigned __int64 b, char* out, size_t sz)
{
    static const char* u[] = {"Б", "КБ", "МБ", "ГБ", "ТБ", "ПБ"};
    double v = (double)b;
    int i = 0;
    while (v >= 1024.0 && i < 5)
    {
        v /= 1024.0;
        i++;
    }
    _snprintf(out, sz, "%I64u байт (%.2f %s)", b, v, u[i]);
    out[sz - 1] = 0;
}

static const char* dow_name(unsigned d)
{
    static const char* n[] = {"воскресенье", "понедельник",
        "вторник", "среда",
        "четверг", "пятница", "суббота"};

```

```

    return d < 7 ? n[d] : "?";
}

static void show_os(record* recs, int n)
{
    for (int i = 0; i < n; i++)
    {
        record* r = &recs[i];
        if (strcmp(r->tag, "OS") != 0)
            continue;
        const char* pt = get(r, "product_type");
        const char* ptr = !strcmp(pt, "workstation") ? "рабочая
станция"
                : !strcmp(pt, "domain_controller")
                ? "контроллер домена"
                : "сервер";
        con_print("Операционная система\n");
        con_print("  Имя компьютера : %s\n", get(r, "computer_name"));
        con_print("  ОС                : %s\n", get(r, "name"));
        con_print("  Product name   : %s\n", get(r, "product_name"));
        if (*get(r, "display_version"))
            con_print("  Версия выпуска : %s\n", get(r,
"display_version"));
        con_print("  Версия          : %s.%s, сборка %s\n", get(r,
"major"),
                get(r, "minor"), get(r, "build"));
        con_print("  Service Pack   : %s (%s.%s)\n",
                *get(r, "service_pack") ? get(r, "service_pack") :
"нет",
                get(r, "sp_major"), get(r, "sp_minor"));
        con_print("  Тип            : %s\n", ptr);
        con_print("  Архитектура    : %s\n", get(r, "arch"));
    }
}

static void show_time(record* recs, int n)
{
    for (int i = 0; i < n; i++)
    {
        record* r = &recs[i];

```

```

    if (strcmp(r->tag, "TIME") != 0)
        continue;
    int local = !strcmp(get(r, "scope"), "local");
    con_print("%s\n", local ? "Местное время сервера" : "Время
UTC");
    con_print("  Дата и время : %04u-%02u-%02u %02u:%02u:%02u.
%03u, %s\n",
        (unsigned)getu(r, "year"), (unsigned)getu(r, "month"),
        (unsigned)getu(r, "day"), (unsigned)getu(r, "hour"),
        (unsigned)getu(r, "minute"), (unsigned)getu(r,
"second"),
        (unsigned)getu(r, "millisecond"),
        dow_name((unsigned)getu(r, "day_of_week")));
    con_print(
        "  Год %s, месяц %s, день %s, часов %s, минут %s, секунд
%s\n",
        get(r, "year"), get(r, "month"), get(r, "day"), get(r,
"hour"),
        get(r, "minute"), get(r, "second"));
    if (local)
    {
        long off = atol(get(r, "utc_offset_minutes"));
        con_print("  Часовой пояс : UTC%c%02ld:%02ld (%s)\n",
            off < 0 ? '-' : '+', labs(off) / 60, labs(off) % 60,
            get(r, "tz_name"));
    }
    else
    {
        con_print("  Unix-время   : %s c\n", get(r, "unix"));
    }
    con_print("  ISO 8601       : %s\n", get(r, "iso"));
}
}

static void show_uptime(record* recs, int n)
{
    for (int i = 0; i < n; i++)
    {
        record* r = &recs[i];
        if (strcmp(r->tag, "UPTIME") != 0)

```

```

        continue;
    unsigned __int64 ms = getu(r, "milliseconds");
    unsigned __int64 s = ms / 1000;
    con_print("Время работы с момента запуска ОС\n");
    con_print("  %I64u дн. %02u ч. %02u мин. %02u с. %03u мс\n",
s / 86400,
        (unsigned)(s / 3600 % 24), (unsigned)(s / 60 % 60),
        (unsigned)(s % 60), (unsigned)(ms % 1000));
    con_print("  Всего: %I64u с = %.2f мин = %.2f ч = %.3f дн.\n",
s,
        (double)ms / 60000.0, (double)ms / 3600000.0,
        (double)ms / 86400000.0);
    con_print("  ОС запущена (UTC): %s\n", get(r,
"boot_time_utc"));
}
}

```

```

static void show_memory(record* recs, int n)
{
    static const struct
    {
        const char *key, *title;
    } rows[] = {
        {"total_phys", "Физическая память, всего"},
        {"avail_phys", "Физическая память, свободно"},
        {"total_pagefile", "Файл подкачки+ОЗУ, всего"},
        {"avail_pagefile", "Файл подкачки+ОЗУ, свободно"},
        {"total_virtual", "Виртуальная память, всего"},
        {"avail_virtual", "Виртуальная память, свободно"},
    };
    for (int i = 0; i < n; i++)
    {
        record* r = &recs[i];
        if (strcmp(r->tag, "MEMORY") != 0)
            continue;
        con_print("Использование памяти\n");
        con_print("  Загрузка памяти: %s %%\n", get(r,
"load_percent"));
        for (int k = 0; k < 6; k++)
        {

```

```

        char b[96];
        fmt_bytes(getu(r, rows[k].key), b, sizeof(b));
        con_print("  %-30s: %s\n", rows[k].title, b);
    }
}

static void show_drives(record* recs, int n)
{
    con_print("Подключённые диски\n");
    con_print("  %-6s %-12s %-10s %-8s %s\n", "Диск", "Тип",
"Класс", "ФС",
        "Метка");
    for (int i = 0; i < n; i++)
    {
        record* r = &recs[i];
        if (strcmp(r->tag, "DRIVE") != 0)
            continue;
        const char* cls = get(r, "class");
        const char* clsr = !strcmp(cls, "local")      ? "локальный"
                        : !strcmp(cls, "network")    ? "сетевой"
                        : !strcmp(cls, "removable") ? "съёмный"
                        : "неизвестно";
        const char* fs = get(r, "fs");
        con_print("  %-6s %-12s %-10s %-8s %s%s\n", get(r, "root"),
            get(r, "type"), clsr, *fs ? fs : "-", get(r, "label"),
            !strcmp(get(r, "ready"), "0") ? " (нет данных/не готов)"
            : "");
    }
}

static void show_freespace(record* recs, int n)
{
    con_print("Свободное место на локальных дисках\n");
    for (int i = 0; i < n; i++)
    {
        record* r = &recs[i];
        if (strcmp(r->tag, "FREESPACE") != 0)
            continue;
        if (*get(r, "error"))

```

```

    {
        con_print("  %s: ошибка %s\n", get(r, "root"), get(r,
"error"));
        continue;
    }
    char t[96], f[96];
    unsigned __int64 tot = getu(r, "total_bytes"),
        fr = getu(r, "free_bytes");
    fmt_bytes(tot, t, sizeof(t));
    fmt_bytes(fr, f, sizeof(f));
    con_print("  %s\n      всего      : %s\n      свободно: %s (%.1f %
%)\n",
        get(r, "root"), t, f,
        tot ? 100.0 * (double)fr / (double)tot : 0.0);
}
}

static const char* scope_text(const char* code, int reg)
{
    if (!strcmp(code, "this_object_only"))
        return reg ? "только этот раздел" : "только этот объект";
    if (!strcmp(code, "this_folder_subfolders_and_files"))
        return reg ? "этот раздел и подразделы"
            : "эта папка, вложенные папки и файлы";
    if (!strcmp(code, "this_folder_and_subfolders"))
        return reg ? "этот раздел и подразделы" : "эта папка и
вложенные папки";
    if (!strcmp(code, "this_folder_and_files"))
        return "эта папка и файлы";
    if (!strcmp(code, "subfolders_and_files_only"))
        return reg ? "только подразделы" : "только вложенные папки и
файлы";
    if (!strcmp(code, "subfolders_only"))
        return reg ? "только подразделы" : "только вложенные папки";
    if (!strcmp(code, "files_only"))
        return "только файлы";
    return code;
}

static void show_ace(const record* r, int reg)

```

```

{
    const char* type = get(r, "ace_type");
    const char* trn = !strcmp(type, "ACCESS_ALLOWED", 14) ?
"разрешающий"
        : !strcmp(type, "ACCESS_DENIED", 13) ? "запрещающий"
        : "прочий";
    const char* use = get(r, "name_use");
    con_print("    ACE #%u\n", (unsigned)getu(r, "index") + 1);
    con_print("    SID субъекта : %s\n", get(r, "sid"));
    if (*get(r, "name"))
        con_print("    Субъект      : %s (%s)\n", get(r, "name"),
use);
    else
        con_print("    Субъект      : <имя не определено> (%s)\n",
use);
    con_print("    Тип ACE        : %s (%s), код %s\n", type, trn,
        get(r, "ace_type_id"));
    con_print("    Флаги ACE      : %s%s%s\n", get(r, "ace_flags"),
        *get(r, "ace_flag_names") ? " = " : "", get(r,
"ace_flag_names"));
    con_print("    Область        : %s%s%s\n", scope_text(get(r,
"scope"), reg),
        strstr(get(r, "ace_flag_names"), "NO_PROPAGATE")
        ? ", не дальше непосредственных потомков"
        : "",
        strstr(get(r, "ace_flag_names"), "INHERITED_ACE")
        ? " [унаследован]"
        : " [задан явно]");
    con_print("    Маска доступа: %s\n", get(r, "mask"));

    char *bits = _strdup(get(r, "bits")), *names = _strdup(get(r,
"bit_names"));
    char *bs = bits, *ns = names;
    con_print("    Биты маски (номер - название):\n");
    while (bs && *bs)
    {
        char *bn = strchr(bs, ','), *nn = strchr(ns, ',');
        if (bn)
            *bn++ = 0;
        if (nn)

```

```

        *nn++ = 0;
    con_print("        %2s  %s\n", bs, ns);
    bs = bn;
    ns = nn ? nn : (char*)"";
}
free(bits);
free(names);
}

static void show_acl(record* recs, int n)
{
    int reg = 0;
    for (int i = 0; i < n; i++)
    {
        record* r = &recs[i];
        if (!strcmp(r->tag, "ACL"))
        {
            const char* ot = get(r, "object_type");
            reg = !strcmp(ot, "registry");
            con_print("Права доступа (DACL)\n");
            con_print("  Объект : %s (%s)\n", get(r, "path"),
                reg ? "ключ реестра"
                : !strcmp(ot, "directory") ? "папка"
                : "файл");
            if (!strcmp(get(r, "null_dacl"), "1"))
                con_print(
                    "    DACL : NULL - доступ разрешён всем без
ограничений\n");
            else
                con_print("  Записей ACE: %s, наследование от родителя:
%s\n",
                    get(r, "ace_count"),
                    !strcmp(get(r, "dacl_protected"), "1") ? "отключено"
                    : "включено");
        }
        else if (!strcmp(r->tag, "ACE"))
        {
            show_ace(r, reg);
        }
    }
}

```

```

}

static void show_owner(record* recs, int n)
{
    for (int i = 0; i < n; i++)
    {
        record* r = &recs[i];
        if (strcmp(r->tag, "OWNER") != 0)
            continue;
        const char* ot = get(r, "object_type");
        con_print("Владелец объекта\n");
        con_print("  Объект : %s (%s)\n", get(r, "path"),
            !strcmp(ot, "registry") ? "ключ реестра"
            : !strcmp(ot, "directory") ? "папка"
            : "файл");
        con_print("  SID      : %s\n", get(r, "sid"));
        con_print("  Имя       : %s (%s)\n",
            *get(r, "name") ? get(r, "name") : "<имя не
определено>",
            get(r, "name_use"));
    }
}

/*
-----
*/
/* Выполнение команды и вывод
*/
/*
-----
*/

static void run_command(const char* cmd, const char* a1, const
char* a2)
{
    sbuf req;
    sb_init(&req);
    sb_append_esc(&req, cmd);
    if (a1)
    {
        sb_append(&req, "\t");
    }
}

```

```

    sb_append_esc(&req, a1);
}
if (a2)
{
    sb_append(&req, "\t");
    sb_append_esc(&req, a2);
}

char* resp = do_request(req.data);
sb_free(&req);
if (!resp)
    return;

if (g_raw)
    con_print("--- сырой ответ ---\n%s--- конец ---\n", resp);

/* разбор строк ответа */
int cap = 64, n = 0;
record* recs = (record*)malloc(sizeof(record) * (size_t)cap);
char* p = resp;
while (*p)
{
    char* eol = strchr(p, '\n');
    if (eol)
        *eol = 0;
    if (*p)
    {
        if (n == cap)
        {
            cap *= 2;
            recs = (record*)realloc(recs, sizeof(record) *
(size_t)cap);
        }
        rec_parse(p, &recs[n++]);
    }
    if (!eol)
        break;
    p = eol + 1;
}

```

```

if (n == 0 || strcmp(recs[0].tag, "RESULT") != 0)
{
    con_print("Некорректный ответ сервера\n");
}
else if (strcmp(get(&recs[0], "status"), "OK") != 0)
{
    con_print("Сервер вернул ошибку %s (%s): %s\n", get(&recs[0],
"code"),
        get(&recs[0], "source"), get(&recs[0], "message"));
}
else if (!strcmp(cmd, "GET_OS"))
    show_os(recs, n);
else if (!strcmp(cmd, "GET_TIME"))
    show_time(recs, n);
else if (!strcmp(cmd, "GET_UPTIME"))
    show_uptime(recs, n);
else if (!strcmp(cmd, "GET_MEMORY"))
    show_memory(recs, n);
else if (!strcmp(cmd, "GET_DRIVES"))
    show_drives(recs, n);
else if (!strcmp(cmd, "GET_FREESPACE"))
    show_freespace(recs, n);
else if (!strcmp(cmd, "GET_ACL"))
    show_acl(recs, n);
else if (!strcmp(cmd, "GET_OWNER"))
    show_owner(recs, n);

free(recs);
free(resp);
}

static void ask_server(void)
{
    char line[300];
    con_print("Адрес сервера (IP или имя, можно host:порт) [%s]: ",
g_host);
    if (!con_readline(line, sizeof(line)) || !line[0])
        return;
    char* colon = strrchr(line, ':');
    if (colon)

```

```

{
    *colon++ = 0;
    int p = atoi(colon);
    if (p > 0 && p < 65536)
        g_port = (unsigned short)p;
}
if (line[0])
{
    strncpy(g_host, line, sizeof(g_host) - 1);
    g_host[sizeof(g_host) - 1] = 0;
}
}

static void ask_object(const char* cmd)
{
    char kind[16], path[1024];
    con_print("Тип объекта: 1 - файл/папка, 2 - ключ реестра [1]:");
    if (!con_readline(kind, sizeof(kind)))
        return;
    int reg = (kind[0] == '2');
    if (reg)
        con_print("Путь к ключу (например HKLM\\SOFTWARE\\Microsoft, "
            "HKCU\\Software): ");
    else
        con_print("Путь к файлу или папке (например C:\\Windows): ");
    if (!con_readline(path, sizeof(path)) || !path[0])
        return;
    run_command(cmd, reg ? "REG" : "FILE", path);
}

int main(void)
{
    WSADATA wsa;
    char line[64];

    con_init();
    if (WSAStartup(MAKEWORD(2, 2), &wsa) != 0)
    {
        con_print("WSAStartup: ошибка\n");
    }
}

```

```

    return 1;
}
if (!crypto_init())
    return 1;

con_print("Генерация пары ключей RSA-2048...\n");
if (!crypto_gen_keypair(&g_pair))
    return 1;
g_pub = crypto_export_pubkey(g_pair, &g_publen);
if (!g_pub)
    return 1;

ask_server();
for (;;)
{
    con_print("\n=== Сервер: %s:%u%s ===\n", g_host, g_port,
        g_raw ? " [сырой вывод]" : "");
    con_print(" 1 Тип и версия ОС\n"
        " 2 Текущее время\n"
        " 3 Время с момента запуска ОС\n"
        " 4 Информация о памяти\n"
        " 5 Типы подключённых дисков и файловые системы\n"
        " 6 Свободное место на локальных дисках\n"
        " 7 Права доступа к файлу/папке/ключу реестра\n"
        " 8 Владелец файла/папки/ключа реестра\n"
        " s Сменить адрес сервера\n"
        " r Вкл/выкл вывод сырого ответа\n"
        " 0 Выход\n"
        "Выбор: ");
    if (!con_readline(line, sizeof(line)))
        break;
    con_print("\n");
    switch (line[0])
    {
    case '1':
        run_command("GET_OS", NULL, NULL);
        break;
    case '2':
        run_command("GET_TIME", NULL, NULL);
        break;

```

```

    case '3':
        run_command("GET_UPTIME", NULL, NULL);
        break;
    case '4':
        run_command("GET_MEMORY", NULL, NULL);
        break;
    case '5':
        run_command("GET_DRIVES", NULL, NULL);
        break;
    case '6':
        run_command("GET_FREESPACE", NULL, NULL);
        break;
    case '7':
        ask_object("GET_ACL");
        break;
    case '8':
        ask_object("GET_OWNER");
        break;
    case 's':
    case 'S':
        ask_server();
        break;
    case 'r':
    case 'R':
        g_raw = !g_raw;
        break;
    case '0':
        goto out;
    default:
        break;
}
}
out:
    free(g_pub);
    crypto_free_key(g_pair);
    crypto_cleanup();
    WSACleanup();
    return 0;
}

```

Приложение Г

common.c

```
#include "common.h"
#include <wincrypt.h>
```

```
/*
```

```
=====
```

```
*/
```

```
/* sbuf
```

```
*/
```

```
/*
```

```
=====
```

```
*/
```

```
static void die_oom(void)
```

```
{
```

```
    fputs("out of memory\n", stderr);
```

```
    exit(1);
```

```
}
```

```
void sb_init(sbuf *b)
```

```
{
```

```
    b->cap = 1024;
```

```
    b->len = 0;
```

```
    b->data = (char *)malloc(b->cap);
```

```
    if (!b->data) die_oom();
```

```
    b->data[0] = 0;
```

```
}
```

```
void sb_free(sbuf *b)
```

```
{
```

```
    free(b->data);
```

```
    b->data = NULL;
```

```
    b->len = b->cap = 0;
```

```
}
```

```
static void sb_reserve(sbuf *b, size_t add)
```

```
{
```

```
    if (b->len + add + 1 > b->cap) {
```

```

        size_t nc = b->cap * 2;
        while (nc < b->len + add + 1) nc *= 2;
        char *p = (char *)realloc(b->data, nc);
        if (!p) die_oom();
        b->data = p;
        b->cap = nc;
    }
}

void sb_append(sbuf *b, const char *s)
{
    size_t n = strlen(s);
    sb_reserve(b, n);
    memcpy(b->data + b->len, s, n + 1);
    b->len += n;
}

void sb_printf(sbuf *b, const char *fmt, ...)
{
    va_list ap, ap2;
    va_start(ap, fmt);
    va_copy(ap2, ap);
    int n = _vscprintf(fmt, ap);
    if (n > 0) {
        sb_reserve(b, (size_t)n);
        _vsnprintf(b->data + b->len, (size_t)n + 1, fmt, ap2);
        b->len += (size_t)n;
        b->data[b->len] = 0;
    }
    va_end(ap2);
    va_end(ap);
}

void sb_append_esc(sbuf *b, const char *s)
{
    for (; *s; s++) {
        switch (*s) {
            case '\\': sb_append(b, "\\"); break;
            case '\t': sb_append(b, "\\t"); break;
            case '\n': sb_append(b, "\\n"); break;

```

```

        case '\\r': sb_append(b, "\\r"); break;
        default: {
            char c[2] = { *s, 0 };
            sb_append(b, c);
        }
    }
}

void sb_rec_begin(sbuf *b, const char *tag) { sb_append(b, tag); }
void sb_rec_end(sbuf *b) { sb_append(b, "\\n"); }

void sb_kv_str(sbuf *b, const char *key, const char *utf8)
{
    sb_append(b, "\\t");
    sb_append(b, key);
    sb_append(b, "=");
    sb_append_esc(b, utf8 ? utf8 : "");
}

void sb_kv_wstr(sbuf *b, const char *key, const wchar_t *w)
{
    char *u = wide_to_utf8(w ? w : L "");
    sb_kv_str(b, key, u);
    free(u);
}

void sb_kv_u64(sbuf *b, const char *key, unsigned __int64 v)
{
    char tmp[32];
    _ui64toa(v, tmp, 10);
    sb_kv_str(b, key, tmp);
}

void sb_kv_i64(sbuf *b, const char *key, __int64 v)
{
    char tmp[32];
    _i64toa(v, tmp, 10);
    sb_kv_str(b, key, tmp);
}

```

```

void sb_kv_hex(sbuf *b, const char *key, unsigned long v)
{
    char tmp[32];
    sprintf(tmp, "0x%08lX", v);
    sb_kv_str(b, key, tmp);
}

/*
-----
*/
/* Разбор записей
*/
/*
-----
*/
void unescape_inplace(char *s)
{
    char *w = s;
    for (char *r = s; *r; r++) {
        if (*r == '\\') {
            r++;
            switch (*r) {
                case 't': *w++ = '\t'; break;
                case 'n': *w++ = '\n'; break;
                case 'r': *w++ = '\r'; break;
                default: *w++ = *r; break; /* \\ и любой другой
СИМВОЛ */
            }
        } else {
            *w++ = *r;
        }
    }
    *w = 0;
}

int rec_parse(char *line, record *r)
{
    r->n = 0;
    r->tag = line;
}

```

```

char *p = strchr(line, '\\t');
if (!p) return 1;
*p++ = 0;
while (p && *p && r->n < REC_MAX_FIELDS) {
    char *next = strchr(p, '\\t');
    if (next) *next++ = 0;
    char *eq = strchr(p, '=');
    if (eq) {
        *eq = 0;
        r->key[r->n] = p;
        r->val[r->n] = eq + 1;
        unescape_inplace(eq + 1);
        r->n++;
    }
    p = next;
}
return 1;
}

const char *rec_get(const record *r, const char *key)
{
    for (int i = 0; i < r->n; i++)
        if (strcmp(r->key[i], key) == 0) return r->val[i];
    return NULL;
}

/*
=====
*/
/* Юникод и консоль
*/
/*
=====
*/
wchar_t *utf8_to_wide(const char *s)
{
    int n = MultiByteToWideChar(CP_UTF8, 0, s, -1, NULL, 0);
    if (n <= 0) return _wcsdup(L "");
    wchar_t *w = (wchar_t *)malloc((size_t)n * sizeof(wchar_t));
    if (!w) die_oom();

```

```

        MultiByteToWideChar(CP_UTF8, 0, s, -1, w, n);
        return w;
    }

char *wide_to_utf8(const wchar_t *w)
{
    int n = WideCharToMultiByte(CP_UTF8, 0, w, -1, NULL, 0, NULL,
NULL);
    if (n <= 0) return _strdup("");
    char *s = (char *)malloc((size_t)n);
    if (!s) die_oom();
    WideCharToMultiByte(CP_UTF8, 0, w, -1, s, n, NULL, NULL);
    return s;
}

static CRITICAL_SECTION g_con_cs;
static HANDLE g_out, g_in;
static BOOL g_out_console, g_in_console;
static int g_con_ready;

void con_init(void)
{
    DWORD m;
    if (g_con_ready) return;
    InitializeCriticalSection(&g_con_cs);
    g_out = GetStdHandle(STD_OUTPUT_HANDLE);
    g_in = GetStdHandle(STD_INPUT_HANDLE);
    g_out_console = GetConsoleMode(g_out, &m);
    g_in_console = GetConsoleMode(g_in, &m);
    g_con_ready = 1;
}

void con_write(const char *utf8)
{
    if (!g_con_ready) con_init();
    EnterCriticalSection(&g_con_cs);
    if (g_out_console) {
        wchar_t *w = utf8_to_wide(utf8);
        DWORD written;
        WriteConsoleW(g_out, w, (DWORD)wcslen(w), &written, NULL);
    }
}

```

```

        free(w);
    } else {
        fwrite(utf8, 1, strlen(utf8), stdout);
        fflush(stdout);
    }
    LeaveCriticalSection(&g_con_cs);
}

void con_print(const char *fmt, ...)
{
    va_list ap, ap2;
    va_start(ap, fmt);
    va_copy(ap2, ap);
    int n = _vscprintf(fmt, ap);
    if (n > 0) {
        char *buf = (char *)malloc((size_t)n + 1);
        if (!buf) die_oom();
        _vsnprintf(buf, (size_t)n + 1, fmt, ap2);
        buf[n] = 0;
        con_write(buf);
        free(buf);
    }
    va_end(ap2);
    va_end(ap);
}

int con_readline(char *out, int outsz)
{
    if (!g_con_ready) con_init();
    if (g_in_console) {
        wchar_t w[1024];
        DWORD n = 0;
        if (!ReadConsoleW(g_in, w, 1023, &n, NULL) || n == 0)
            return 0;
        while (n > 0 && (w[n - 1] == L'\\r' || w[n - 1] == L'\\n'))
            n--;
        w[n] = 0;
        if (WideCharToMultiByte(CP_UTF8, 0, w, -1, out, outsz,
            NULL, NULL) == 0)
            out[0] = 0;
    }
}

```

```

        return 1;
    }
    if (!fgets(out, outsz, stdin)) return 0;
    size_t l = strlen(out);
    while (l > 0 && (out[l - 1] == '\n' || out[l - 1] == '\r'))
out[--l] = 0;
    return 1;
}

/*
=====
*/
/* CryptoAPI
*/
/*
=====
*/
static HCRYPTPROV      g_prov;
static CRITICAL_SECTION g_crypto_cs;

static void crypto_err(const char *what)
{
    con_print("CryptoAPI: %s завершилась ошибкой 0x%08lX\n", what,
(unsigned long)GetLastError());
}

int crypto_init(void)
{
    InitializeCriticalSection(&g_crypto_cs);
    /* VERIFYCONTEXT: контейнер ключей на диске не нужен, ключи
эфемерные */
    if (!CryptAcquireContextW(&g_prov, NULL,
MS_ENH_RSA_AES_PROV_W, PROV_RSA_AES,
CRYPT_VERIFYCONTEXT)) {
        crypto_err("CryptAcquireContext");
        return 0;
    }
    return 1;
}

```

```

void crypto_cleanup(void)
{
    if (g_prov) CryptReleaseContext(g_prov, 0);
    g_prov = 0;
}

void crypto_free_key(HCRYPTKEY k)
{
    if (!k) return;
    EnterCriticalSection(&g_crypto_cs);
    CryptDestroyKey(k);
    LeaveCriticalSection(&g_crypto_cs);
}

/* ----- клиент ----- */
int crypto_gen_keypair(HCRYPTKEY *pair)
{
    int ok;
    EnterCriticalSection(&g_crypto_cs);
    ok = CryptGenKey(g_prov, AT_KEYEXCHANGE, (2048u << 16) |
CRYPT_EXPORTABLE, pair);
    if (!ok) crypto_err("CryptGenKey(RSA)");
    LeaveCriticalSection(&g_crypto_cs);
    return ok;
}

BYTE *crypto_export_pubkey(HCRYPTKEY pair, DWORD *len)
{
    BYTE *blob = NULL;
    DWORD n = 0;
    EnterCriticalSection(&g_crypto_cs);
    if (CryptExportKey(pair, 0, PUBLICKEYBLOB, 0, NULL, &n) &&
        (blob = (BYTE *)malloc(n)) != NULL) {
        if (!CryptExportKey(pair, 0, PUBLICKEYBLOB, 0, blob, &n))
        {
            crypto_err("CryptExportKey(PUBLICKEYBLOB)");
            free(blob);
            blob = NULL;
        } else {
            *len = n;
        }
    }
}

```

```

    }
} else if (!blob) {
    crypto_err("CryptExportKey(PUBLICKEYBLOB)");
}
LeaveCriticalSection(&g_crypto_cs);
return blob;
}

int crypto_import_session_key(HCRYPTKEY pair, const BYTE *blob,
DWORD len, HCRYPTKEY *sess)
{
    int ok;
    EnterCriticalSection(&g_crypto_cs);
    /* SIMPLEBLOB расшифровывается приватным ключом пары */
    ok = CryptImportKey(g_prov, blob, len, pair, 0, sess);
    if (!ok) crypto_err("CryptImportKey(SIMPLEBLOB)");
    LeaveCriticalSection(&g_crypto_cs);
    return ok;
}

/* ----- сервер ----- */
int crypto_create_session(const BYTE *pubblob, DWORD publen,
HCRYPTKEY *sess, BYTE **outblob, DWORD
*outlen)
{
    HCRYPTKEY pub = 0, key = 0;
    BYTE *blob = NULL;
    DWORD blen = 0;
    int ok = 0;

    EnterCriticalSection(&g_crypto_cs);
    if (!CryptImportKey(g_prov, pubblob, publen, 0, 0, &pub)) {
        crypto_err("CryptImportKey(PUBLICKEYBLOB)");
        goto done;
    }
    /* случайный сеансовый ключ AES-256 (режим CBC, дополнение
PKCS#5 - по умолчанию) */
    if (!CryptGenKey(g_prov, CALG_AES_256, CRYPT_EXPORTABLE,
&key)) {
        crypto_err("CryptGenKey(AES)");
    }
}

```

```

        goto done;
    }
    /* шифруем сеансовый ключ публичным ключом клиента */
    if (!CryptExportKey(key, pub, SIMPLEBLOB, 0, NULL, &blen)) {
        crypto_err("CryptExportKey(SIMPLEBLOB)");
        goto done;
    }
    blob = (BYTE *)malloc(blen);
    if (!blob) goto done;
    if (!CryptExportKey(key, pub, SIMPLEBLOB, 0, blob, &blen)) {
        crypto_err("CryptExportKey(SIMPLEBLOB)");
        goto done;
    }
    *sess = key;
    *outblob = blob;
    *outlen = blen;
    key = 0;
    blob = NULL;
    ok = 1;
done:
    if (pub) CryptDestroyKey(pub);
    if (key) CryptDestroyKey(key);
    free(blob);
    LeaveCriticalSection(&g_crypto_cs);
    return ok;
}

/* ----- симметричное шифрование ----- */
BYTE *crypto_encrypt(HCRYPTKEY k, const BYTE *data, DWORD len,
DWORD *outlen)
{
    BYTE *out = (BYTE *)malloc(AES_BLOCK + len + AES_BLOCK);
    DWORD n = len;
    int ok = 0;
    if (!out) return NULL;

    EnterCriticalSection(&g_crypto_cs);
    /* для каждого сообщения - новый случайный IV, он передаётся
перед шифртекстом */
    if (CryptGenRandom(g_prov, AES_BLOCK, out) &&

```

```

        CryptSetKeyParam(k, KP_IV, out, 0)) {
        memcpy(out + AES_BLOCK, data, len);
        ok = CryptEncrypt(k, 0, TRUE, 0, out + AES_BLOCK, &n, len
+ AES_BLOCK);
    }
    if (!ok) crypto_err("CryptEncrypt");
    LeaveCriticalSection(&g_crypto_cs);

    if (!ok) {
        free(out);
        return NULL;
    }
    *outlen = AES_BLOCK + n;
    return out;
}

```

```

BYTE *crypto_decrypt(HCRYPTKEY k, const BYTE *data, DWORD len,
DWORD *outlen)
{
    if (len < 2 * AES_BLOCK || (len % AES_BLOCK) != 0) return
NULL;
    DWORD n = len - AES_BLOCK;
    BYTE *out = (BYTE *)malloc((size_t)n + 1);
    int ok;
    if (!out) return NULL;
    memcpy(out, data + AES_BLOCK, n);

    EnterCriticalSection(&g_crypto_cs);
    ok = CryptSetKeyParam(k, KP_IV, (BYTE *)data, 0) &&
        CryptDecrypt(k, 0, TRUE, 0, out, &n);
    LeaveCriticalSection(&g_crypto_cs);

    if (!ok) {
        free(out);
        return NULL;
    }
    out[n] = 0;
    *outlen = n;
    return out;
}

```

```

/*
=====
*/
/* Кадры
*/
/*
=====
*/

DWORD frame_be32(const BYTE *p)
{
    return ((DWORD)p[0] << 24) | ((DWORD)p[1] << 16) |
    ((DWORD)p[2] << 8) | (DWORD)p[3];
}

static void put_be32(BYTE *p, DWORD v)
{
    p[0] = (BYTE)(v >> 24);
    p[1] = (BYTE)(v >> 16);
    p[2] = (BYTE)(v >> 8);
    p[3] = (BYTE)v;
}

BYTE *frame_wrap(const BYTE *body, DWORD len, DWORD *outlen)
{
    BYTE *f = (BYTE *)malloc((size_t)len + 4);
    if (!f) return NULL;
    put_be32(f, len);
    memcpy(f + 4, body, len);
    *outlen = len + 4;
    return f;
}

static int send_all(SOCKET s, const BYTE *p, int n)
{
    while (n > 0) {
        int r = send(s, (const char *)p, n, 0);
        if (r <= 0) return 0;
        p += r;
        n -= r;
    }
}

```

```

    }
    return 1;
}

static int recv_all(SOCKET s, BYTE *p, int n)
{
    while (n > 0) {
        int r = recv(s, (char *)p, n, 0);
        if (r <= 0) return 0;
        p += r;
        n -= r;
    }
    return 1;
}

int net_send_frame(SOCKET s, const BYTE *data, DWORD len)
{
    BYTE hdr[4];
    put_be32(hdr, len);
    return send_all(s, hdr, 4) && send_all(s, data, (int)len);
}

BYTE *net_recv_frame(SOCKET s, DWORD maxlen, DWORD *outlen)
{
    BYTE hdr[4];
    if (!recv_all(s, hdr, 4)) return NULL;
    DWORD len = frame_be32(hdr);
    if (len == 0 || len > maxlen) return NULL;
    BYTE *p = (BYTE *)malloc(len);
    if (!p) return NULL;
    if (!recv_all(s, p, (int)len)) {
        free(p);
        return NULL;
    }
    *outlen = len;
    return p;
}

```

Приложение Д

common.h

```
#ifndef COMMON_H
#define COMMON_H

#ifndef WIN32_LEAN_AND_MEAN
#define WIN32_LEAN_AND_MEAN
#endif

#ifndef _WIN32_WINNT
#define _WIN32_WINNT 0x0601 /* Windows 7 */
#endif

#ifndef _CRT_SECURE_NO_WARNINGS
#define _CRT_SECURE_NO_WARNINGS
#endif

#include <winsock2.h>
#include <windows.h>
#include <wincrypt.h>
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <stdarg.h>

#define PROTO_PORT 9000
#define SERVER_RX_MAX 16384 /* макс. размер кадра, принимаемого
сервером */
#define CLIENT_RX_MAX
\
(4 * 1024 * 1024) /* макс. размер кадра, принимаемого клиентом
*/
#define AES_BLOCK 16

/*
-----
*/
/* Строковый буфер и записи протокола
*/
/*
```

```

-----
*/
typedef struct
{
    char* data;
    size_t len;
    size_t cap;
} sbuf;

void sb_init(sbuf* b);
void sb_free(sbuf* b);
void sb_append(sbuf* b, const char* s);
void sb_printf(sbuf* b, const char* fmt, ...);
void sb_append_esc(sbuf* b, const char* s); /* добавить с
экранированием */

/* запись: TAG \t key=value \t key=value ... \n */
void sb_rec_begin(sbuf* b, const char* tag);
void sb_kv_str(sbuf* b, const char* key, const char* utf8);
void sb_kv_wstr(sbuf* b, const char* key, const wchar_t* w);
void sb_kv_u64(sbuf* b, const char* key, unsigned __int64 v);
void sb_kv_i64(sbuf* b, const char* key, __int64 v);
void sb_kv_hex(sbuf* b, const char* key, unsigned long v);
void sb_rec_end(sbuf* b);

/* разбор записи (на стороне клиента и при разборе запроса на
сервере) */
#define REC_MAX_FIELDS 40
typedef struct
{
    char* tag;
    int n;
    char* key[REC_MAX_FIELDS];
    char* val[REC_MAX_FIELDS];
} record;

void unescape_inplace(char* s);
int rec_parse(char* line, record* r); /* line портится
(разрезается) */
const char* rec_get(const record* r, const char* key);

```

```

/*
-----
*/
/* Юникод и консоль
*/
/*
-----
*/
wchar_t* utf8_to_wide(const char* s); /* free() после
использования */
char* wide_to_utf8(const wchar_t* w);

void con_init(void);
void con_write(const char* utf8);
void con_print(const char* fmt, ...); /* printf, но строки в UTF-8
*/
int con_readline(char* out, int outsz); /* 0 - конец ввода */

/*
-----
*/
/* CryptoAPI
*/
/*
-----
*/
int crypto_init(void);
void crypto_cleanup(void);
void crypto_free_key(HCRYPTKEY k);

/* клиентская часть */
int crypto_gen_keypair(HCRYPTKEY* pair);
BYTE* crypto_export_pubkey(HCRYPTKEY pair, DWORD* len);
int crypto_import_session_key(HCRYPTKEY pair, const BYTE* blob,
DWORD len,
HCRYPTKEY* sess);

/* серверная часть: импорт публичного ключа клиента, генерация и
экспорт

```

```

    * сеансового ключа */
int crypto_create_session(const BYTE* pubblob, DWORD publen,
    HCRYPTKEY* sess,
        BYTE** outblob, DWORD* outlen);

/* обе стороны. Результат encrypt: IV(16) || шифртекст. decrypt
возвращает буфер
    * с завершающим 0 */
BYTE* crypto_encrypt(HCRYPTKEY k, const BYTE* data, DWORD len,
    DWORD* outlen);
BYTE* crypto_decrypt(HCRYPTKEY k, const BYTE* data, DWORD len,
    DWORD* outlen);

/*
-----
*/
/* Кадры: 4 байта длины (big endian) + тело
*/
/*
-----
*/
BYTE* frame_wrap(const BYTE* body, DWORD len,
    DWORD* outlen); /* для асинхронной отправки */
DWORD frame_be32(const BYTE* p);

/* блокирующие варианты (клиент) */
int net_send_frame(SOCKET s, const BYTE* data, DWORD len);
BYTE* net_rcv_frame(SOCKET s, DWORD maxlen, DWORD* outlen);

/*
-----
*/
/* sysinfo.c
*/
/*
-----
*/
/* Разобрать запрос и сформировать ответ. Возвращает 1 при успехе,
0 при ошибке.
*/

```

```
int process_request(const char* req, sbuf* resp);  
/* Короткое читаемое описание запроса для журнала сервера */  
void request_summary(const char* req, char* out, size_t outsz);  
  
#endif
```